Pimcore Developer Book

A comprehensive, chapter-wise guide to building production-grade applications with Pimcore — from first install to Kubernetes deployment and open-source contribution.

Table of Contents

#	Chapter	Topics Covered
01	**Introduction to Pimcore**	What is Pimcore, modules (PIM/DAM/CMS/MDM/eCommerce), core building blocks, architecture
02	**Setup & Installation**	System requirements, Composer install, Docker setup, database setup, common pitfalls
03	**Data Objects & Classes**	Classes, Data Objects, Object Bricks, Field Collections, Localized Fields, Relations, PHP API
04	**Assets & Digital Asset Management**	Upload, metadata, tagging, image thumbnails, connecting assets to objects, CDN strategy
05	**CMS Documents**	Pages, Snippets, Emails, editable areas in Twig, organizing content, headless vs. traditional
06	**Controllers & Routing**	Symfony controllers in Pimcore, product list/detail/category/search, pagination, card grids
07	**REST API & GraphQL**	Custom REST endpoints, pagination & filtering, DataHub GraphQL, comparing approaches
08	**Workflows & Localization**	Workflow states/transitions/roles, YAML config, multilingual fields, language trees
09	**eCommerce & Product Catalog**	eCommerce framework, product index, catalog pages, related products, cart & checkout
10	**Bundles & Extensions**	Custom bundles, bundle structure, services & controllers, Admin UI extensions (ExtJS)

#	Chapter	Topics Covered
11	**Event Subscribers & Hooks**	Pimcore events, auto-slug generation, audit logging, external sync on save
12	**Data Import & Export**	CSV import command, image downloading, export to CSV, dry-run mode, error handling
13	**Performance & Caching**	OPcache, Redis, Varnish/Nginx full-page cache, MySQL tuning, Elasticsearch, profiling
14	**Security & Permissions**	Users & roles, object/asset permissions, Symfony firewall, JWT, auditing
15	**Debugging & Troubleshooting**	Log files, Symfony profiler, SQL debugging, common errors, Xdebug, debug commands
16	**Testing**	PHPUnit setup, unit tests, functional controller tests, CI/CD with GitHub Actions
17	**Deployment & CI/CD**	Env vars, class definitions, migration sequence, zero-downtime deploy, GitHub Actions
18	**Kubernetes Setup & Debugging**	K8s architecture, Dockerfiles, manifests (Deployments/Services/PVC/Ingress), HPA, debugging pods
19	**Contributing to Pimcore**	Finding issues, fork & branch, PR process, coding style
20	**Learning Roadmap**	5-week hands-on plan from zero to PR contribution

---

How to Read This Book

If you're brand new to Pimcore: Read chapters 1 → 2 → 3 → 4 → 5 in order. This gives you the core mental model.

If you're building a product catalog: Read 3 → 4 → 6 → 9 → 7.

If you're adding APIs: Read 7 → 8 → 11.

If you're going to production: Read 13 → 14 → 15 → 17 → 18.

If you want to extend Pimcore: Read 10 → 11 → 12.

Key Concepts at a Glance

```
Pimcore
├── Data Layer
│   ├── Classes (blueprints/schemas)
│   ├── Data Objects (records/instances)
│   ├── Assets (images, videos, PDFs)
│   └── Documents (CMS pages, snippets, emails)
│
├── Application Layer (Symfony)
│   ├── Controllers (routing + responses)
│   ├── Services (business logic)
│   ├── Bundles (reusable feature packages)
│   └── Event Subscribers (hooks)
│
├── API Layer
│   ├── Custom REST (Symfony controllers)
│   └── GraphQL (DataHub bundle)
│
└── Infrastructure
    ├── MySQL (data store)
    ├── Redis (cache + sessions)
    ├── Elasticsearch (search + product index)
    └── Kubernetes (container orchestration)
```

Chapter 1: Introduction to Pimcore

What is Pimcore?

Pimcore is an **open-source Digital Experience Platform (DXP)** built on the **Symfony PHP framework**. It is a single, unified platform that helps businesses manage, enrich, and deliver digital data across multiple channels — websites, mobile apps, marketplaces, print, and IoT devices.

Instead of running five separate tools (one for product data, one for images, one for content, one for eCommerce), Pimcore brings everything under one roof.

> **Analogy:** Think of Pimcore as a "digital warehouse + factory":
>
> - **Warehouse shelves** (Data Objects & Assets) store your products, images, and content.
> - **Blueprints** (Classes) define how things are structured.
> - **Factory machines** (Admin UI + APIs) let you organize, enrich, and distribute data anywhere.

Main Modules

1. PIM — Product Information Management

- Central hub for all product-related data.
- Manages attributes (size, color, material, SKU), variants, categories, and relationships.
- Ensures consistent product data across websites, apps, and marketplaces.

2. DAM — Digital Asset Management

- Central repository for all media assets (images, videos, PDFs, etc.).
- Provides metadata management, version control, tagging, and rights management.
- Assets can be directly linked to products, categories, or content pages.

3. CMS — Content Management System

- Manages website content (pages, blogs, landing pages).
- WYSIWYG editing and drag-and-drop layouts.
- Supports both traditional CMS and **headless CMS** approaches.

4. MDM — Master Data Management

- Extends PIM to handle all types of data (customers, suppliers, vendors, locations).
- Ensures a single source of truth for critical business data.

5. eCommerce Framework

- Flexible toolkit to create product catalogs, carts, and checkout processes.

- Can integrate with ERP, CRM, and payment systems.
- Supports personalized shopping experiences.

Core Building Blocks

Building Block	What It Is	Example
Data Objects	Structured business entities	A `Product` with name, SKU, price
Classes	Schema/blueprint that defines Object structure	The `Product` class with field definitions
Assets	Raw files (images, videos, PDFs)	`laptop.jpg`, `manual.pdf`
Documents	CMS pages, snippets, email templates	`/about-us` page, product detail page
Admin UI	ExtJS-powered backend for editors and admins	The `/admin` dashboard

> **Rule of thumb:**
>
> - Objects = structured data
> - Assets = media files
> - Documents = content pages

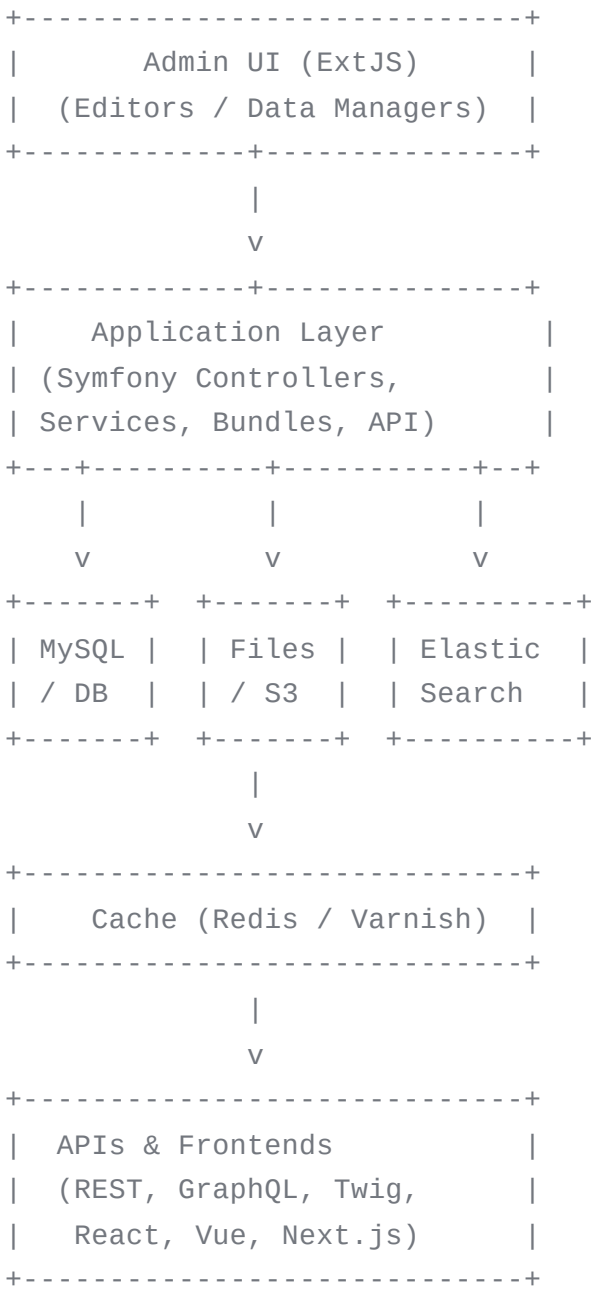
Problems Pimcore Solves

Problem	Solution
Data chaos	Unifies scattered product, asset, and content data
Content silos	Single-source publishing across websites, apps, marketplaces, and print

Problem	Solution
Integration pain	API-first and Symfony-based, plugs into modern stacks
Scalability	Handles millions of products, files, or pages efficiently

---

Architecture Overview



Real-World Use Cases

- **PIM:** Manage product catalogs, specs, attributes, multi-language data.
- **MDM:** Single source of truth for enterprise data.
- **DAM:** Organize, transform, and deliver digital assets.
- **CMS / Headless CMS:** Manage websites or deliver structured content to apps via APIs.
- **eCommerce:** Extend with product management, carts, pricing, personalization.

Key Technologies Under the Hood

- **PHP 8.1+** — runtime
- **Symfony** — framework (routing, DI, events, security)
- **MySQL / MariaDB** — primary data store
- **Redis** — caching and sessions (recommended)
- **Elasticsearch / OpenSearch** — search and product indexing (optional but critical for large catalogs)
- **Twig** — templating engine
- **ExtJS** — Admin UI framework
- **Composer** — dependency management

Chapter 2: Setup & Installation

System Requirements

Before installing Pimcore, make sure your environment meets these requirements:

Requirement	Minimum Version
PHP	≥ 8.1 (8.3 recommended for latest Pimcore)
MySQL	≥ 8.0
MariaDB	≥ 10.4

Requirement	Minimum Version
Composer	Latest
Node.js / npm	Optional (for front-end assets)
Web Server	Apache or Nginx

Option A: Using Composer (Recommended for production)

```
# Create a new project (skeleton = empty starter)
composer create-project pimcore/skeleton my-project

# OR use the demo project (has sample data)
composer create-project pimcore/demo my-project

# Move into the project folder
cd my-project

# Install dependencies
composer install
```

Option B: Using Docker (Easiest for beginners)

Step 1: Install Docker + Docker Compose

On Ubuntu/Debian:

```
# Install dependencies
sudo apt update
sudo apt install -y ca-certificates curl gnupg lsb-release

# Add Docker GPG key and repo
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

```
echo \
  "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

# Allow your user to run docker without sudo
sudo usermod -aG docker $USER
newgrp docker
```

Verify:

```
docker run --rm hello-world
docker compose version
```

Step 2: Create a Pimcore Project Using the Official Image

```
mkdir ~/pimcore && cd ~/pimcore

# Use the Pimcore Docker image to run Composer (no PHP needed locally)
docker run -u $(id -u):$(id -g) --rm -v $(pwd):/var/www/html \
  pimcore/pimcore:php8.3-latest composer create-project pimcore/demo my-project

cd my-project
```

> **Why `-u $(id -u):$(id -g)`?** This ensures files created inside Docker are owned by your user, preventing permission headaches.

Step 3: Configure docker-compose.yaml

```
# Check your UID:GID
echo $(id -u):$(id -g)

# Open docker-compose.yaml and uncomment the `user:` lines
# Replace with your uid:gid (e.g., user: "1000:1000")
```

Step 4: Start the Stack

```
docker compose up -d

# Check running containers
docker compose ps
```

Step 5: Run the Pimcore Installer

```
docker compose exec php vendor/bin/pimcore-install \
  --admin-username=admin --admin-password=admin \
  --mysql-username=pimcore --mysql-password=pimcore --mysql-database=pimcore \
  --mysql-host-socket=db --no-interaction
```

> This creates the database schema, admin user, and all base configuration.

Step 6: Access Pimcore

- **Frontend:** `http://localhost`
- **Admin UI:** `http://localhost/admin`
- **Login:** Use the credentials you set in the installer (e.g., `admin` / `admin`)

Useful Development Commands

```
# Follow logs
docker compose logs -f php
docker compose logs -f db

# Open a shell inside the PHP container
docker compose exec php bash

# Clear Symfony/Pimcore cache
docker compose exec php bin/console cache:clear

# Check system requirements
docker compose exec php bin/console pimcore:system:requirements:check

# Stop and remove containers + volumes (clean slate)
docker compose down -v --remove-orphans
```

Common Pitfalls & Fixes

1. Permission Errors (files owned by root)

```
# On host
sudo chown -R $(id -u):$(id -g) .

# Or inside container
docker compose exec php bash -lc "chown -R www-data:www-data var public vendor"
```

2. Database Connection Error (SQLSTATE[HY000] [2002])

- Check that the `db` container is running: `docker compose ps`
- Ensure your DSN in `.env` uses `db` as the host (that's the Docker network hostname)
- Check DB logs: `docker compose logs db`

3. Admin Page 404 After Install

- Ensure your web server document root points to `my-project/public`
- Run `composer install` if you set up without Docker
- Check file permissions on `var/` and `public/var/`

4. `docker compose` Not Found

- Install the Docker Compose plugin: `sudo apt install docker-compose-plugin`

5. Elasticsearch Issues (Demo Project)

- The demo project may require Elasticsearch for search indexing
- You can disable the search backend locally until needed
- Add an `elasticsearch` service to `docker-compose.yaml` if required

Production Architecture (Services Needed)

Service	Purpose
PHP-FPM + Nginx/Apache	Web server + runtime
MySQL / MariaDB	Primary data store
Redis	Cache + sessions (highly recommended)
Elasticsearch	Search + product indexing (for large catalogs)
S3 / NFS	Scalable asset storage
RabbitMQ / Symfony Messenger	Async task queue (optional)
Varnish / CDN	Full-page caching + global delivery
Grafana + Prometheus	Monitoring

Quick Troubleshooting Cheatsheet

```
docker compose ps                    # Check containers
docker compose logs -f php           # Check PHP/Pimcore errors
docker compose exec php vendor/bin/pimcore-install --help  # Installer options
docker compose exec php bin/console cache:clear             # Clear cache
docker compose down -v               # Destroy volumes (fresh start)
```

Chapter 3: Data Objects & Classes

Overview

Data Objects and Classes are the backbone of Pimcore's data modeling system. Think of it like a database but with a visual editor:

- **Class** = the schema/blueprint (like a table definition)
- **Data Object** = the actual record (like a row in that table)

Core Concepts

1. Classes

A **Class** defines the structure (fields and layout) of a Data Object type.

- Created in Admin UI → Settings → Data Objects → Classes
- Each class generates a PHP model automatically (e.g., `Pimcore\Model\DataObject\Product`)
- Example: A `Product` class with fields: `name`, `sku`, `price`, `description`, `image`, `categories`

2. Data Objects

Data Objects are instances created from a Class — the actual content/records.

- Stored in the Data Objects tree (left sidebar in Admin)
- Example: A `Product` object = "iPhone 16 Pro" with SKU `IP16P-256`, price `₹120,000`

3. Object Bricks

Object Bricks are reusable sets of optional fields that can be attached to Classes dynamically.

- Use when: optional or type-specific attributes that not every object has
- Example: An `ElectronicsSpecs` brick with `batteryLife`, `screenSize` — only attached to electronic products, not clothing

4. Field Collections

Field Collections are groups of fields you can repeat multiple times within one object.

- Use when: you need repeatable structured data
- Example: A `TechnicalSpecs` field collection with `label` + `value`:
  - (label=Battery, value=24h)
  - (label=Weight, value=180g)

5. Localized Fields

Localized Fields are a special container for multi-language content.

- Use when: you need fields translated per language

- Example: `name[en]` = "iPhone 16", `name[de]` = "iPhone 16", `name[fr]` = "iPhone 16"

6. Relations

Relations link objects, assets, or documents to each other.

Type	Use Case
Object Relation (Many-to-One)	Product → Category
Asset Relation	Product → Image
Many-to-Many Object Relation	Product → multiple Tags
Many-to-Many Asset Relation	Product → multiple Images

Creating a Product Class (Admin UI — Click by Click)

Step 1: Open Class Editor

1. Log in → Settings (gear icon) → Data Objects → Classes
2. Click **Add** → Choose **Class** → Enter Name: `Product` → Click **Create**

Step 2: Add Fields

Inside the class editor, add these fields:

Field	Type	Notes
`sku`	Input	Mark as unique
`name`	Input	Product name
`price`	Numeric	Set min/max optional
`description`	WYSIWYG or Textarea	Rich text
`images`	Many-to-Many Asset Relation	Allow `image` type assets

Field	Type	Notes
`categories`	Many-to-Many Object Relation	Allow `Category` class

> Tip: You can organize fields into **Panels**, **Fieldsets**, and **Tabs** for better Admin UI layout.

Step 3: Save & Publish the Class

Click **Save & Publish** — Pimcore generates the PHP model and updates the database.

Creating Product Objects

Via Admin UI

1. Go to **Data Objects** in the sidebar
2. Right-click → **Add Object** → `Product`
3. Fill in fields (Name, SKU, Price, etc.)
4. Upload an image from Assets and link it
5. Click **Save & Publish**

Via PHP (Programmatic)

```php
<?php
use Pimcore\Model\DataObject;
use Pimcore\Model\Asset;
use Pimcore\Model\Element\Service as ElementService;

// 1) Ensure parent folder exists
$parent = DataObject\Folder::getByPath('/products');
if (!$parent) {
    $parent = new DataObject\Folder();
    $parent->setKey('products');
    $parent->setParentId(1); // root
    $parent->save();
}

// 2) Create product object
$product = new DataObject\Product();
```

```php
$product->setKey(ElementService::getValidKey('iPhone-16-pro', 'object'));
$product->setParentId($parent->getId());
$product->setPublished(true);

// 3) Set fields (setters follow field name in camelCase)
$product->setSku('IP16P-256');
$product->setName('iPhone 16 Pro');
$product->setPrice(120000);
$product->setDescription('Flagship phone — example description.');

// 4) Attach images (Many-to-Many asset relation = array of Assets)
$imageIds = [123, 124];
$assets = array_filter(array_map(fn($id) => Asset::getById($id), $imageIds));
$product->setImages(array_values($assets));

// 5) Attach categories (Many-to-Many object relation = array of Objects)
$categoryIds = [10, 11];
$cats = array_filter(array_map(fn($id) => DataObject\Category::getById($id),
$categoryIds));
$product->setCategories(array_values($cats));

// 6) Save
$product->save(['versionNote' => 'Created via script']);
echo "Saved product ID: " . $product->getId();
```

Deploying Classes Between Environments

The recommended workflow for moving class definitions between dev → staging → production:

```
# Export class definition to JSON (from Admin UI: Classes → select → Export
JSON)
# OR via console:
php bin/console pimcore:definition:import:class /path/to/Product.json

# Rebuild classes & update DB schema
php bin/console pimcore:deployment:classes-rebuild --create-classes

# Clear cache
php bin/console cache:clear
```

> Always commit class JSON files to Git (`config/pimcore/classes/Product.json`).

Common Pitfalls

Problem	Fix
`Class not found` when using `DataObject\Product`	Run `pimcore:deployment:classes-rebuild --create-classes`
Setter not working	Setter names follow field name exactly in camelCase — `sku` → `setSku()`
Object saved in wrong location	Check `setParentId()` — `1` is the root
Single image vs. array	`Image` field type = single `Asset` instance; Many-to-Many = array of Assets
Field renamed, data lost	Renaming fields can lose stored data — be careful

---

Data Modeling Walkthrough: Products with Variants

A complete example showing all building blocks together:

1. **Create Class:** `Product` → fields: `sku`, `name`, `price`
2. **Add Localized Fields:** for `name` and `description` (multi-language)
3. **Add Object Relation:** to `Category` class
4. **Add Asset Relation:** for `images`
5. **Add Object Brick:** `ElectronicsSpecs` → `batteryLife`, `screenSize`
6. **Add Field Collection:** `TechnicalSpecs` → `label` + `value` pairs
7. **Create Data Objects:**
   - "iPhone 16" → has images + `ElectronicsSpecs`
   - "iPhone 16 Pro" → variant with higher price + extra specs

> **Classes define the structure. Data Objects are the actual records. Object Bricks, Field Collections, and Localized Fields make it flexible. Relations tie everything together.**

Chapter 4: Assets & Digital Asset Management (DAM)

What are Assets?

Assets are all the media files in Pimcore — images, videos, PDFs, documents, audio, and any binary file. They are stored in Pimcore's **Digital Asset Management (DAM)** system, organized in a hierarchical folder tree just like a file system.

Every asset is a Pimcore element with:

- A unique ID
- Metadata fields (alt text, copyright, license, custom fields)
- Version history (Pimcore auto-saves older versions when you re-upload)
- Permission controls (role-based access)

Step 1: Upload Assets

1. In Admin Panel → **Assets** (left sidebar)
2. Right-click → **Add Folder** (e.g., `Product Images`)
3. Inside the folder, **drag & drop** files from your computer

> You can upload: `laptop.jpg`, `manual.pdf`, `promo.mp4`, etc.

Step 2: Metadata & Tagging

Each asset can have custom metadata:

1. Select an uploaded asset (e.g., `laptop.jpg`)
2. Go to the **Properties / Metadata** tab
3. Add fields:
    - `Alt Text` = "Laptop X1 Product Image"
    - `Copyright` = "© Company 2025"
    - `Tags` = Laptop, Electronics, Product

> Metadata makes assets searchable and reusable across projects.

Step 3: Organize Assets

Best practices for large media libraries:

Practice	Why
Use logical folder hierarchies	`/products/electronics/phones` avoids dumping everything in `/`
Define required metadata	Enforce license, usage rights fields
Use tags for quick search	Find all "PDF manuals" in one click
Use versions	Pimcore keeps older versions when you re-upload
Enable CDN	Serve assets through CDN for better performance
Use Image Thumbnails	Never serve original huge images directly
Use S3 / MinIO for huge files	Videos, CAD models belong in object storage

---

Step 4: Image Thumbnails

Pimcore has a powerful **Thumbnail** system that lets you define image transformation presets.

Define a Thumbnail in Admin UI

1. Go to **Settings** → **Thumbnails**
2. Click **Add** → name it `product_preview`
3. Add transformations:
   - Resize to 800×600
   - Crop center
   - Output format: **WebP** (best for web)

4. Save

Use in Twig Templates

```twig
<img src="{{ asset.thumbnail('product_preview') }}" alt="{{ product.name }}">
```

Use in PHP

```php
$asset = \Pimcore\Model\Asset\Image::getById(123);

// Use a named preset
$thumbPath = $asset->getThumbnail('product_preview')->getPath();
echo $thumbPath;

// Or define on-the-fly (not stored in settings)
$thumbConfig = new \Pimcore\Model\Asset\Image\Thumbnail\Config();
$thumbConfig->addItem('resize', ['width' => 400, 'height' => 300, 'forceResize'
=> true]);
$thumbConfig->setFormat('webp');
$thumbConfig->setQuality(85);

$thumbPath = $asset->getThumbnail($thumbConfig)->getPath();
```

Step 5: Connect Assets to Data Objects

Method A: Asset Field in Product Class

1. Go to Settings → Data Objects → Classes → `Product`
2. Add fields:
   - `Image` → **Image** type (for one main image)
   - `Gallery` → **Image Gallery** (for multiple images)
   - `Documents` → **Many-to-Many Object Relation** with Asset (for PDFs, videos)
3. Save & Publish the class

Now open a Product object and drag & drop assets directly into the image fields.

Method B: Check Usage from the Asset Side

1. Open an Asset (e.g., `laptop.jpg`)
2. Go to the **Relations** tab
3. See which Products, Pages, or Objects are using this asset

---

Serving Large Media Libraries

Strategy	Detail
Always use thumbnails	Never serve full-resolution originals to the browser
Different sizes for different contexts	Small `100×100` for grid preview, `1200×800` for web display
Use WebP/AVIF	Significantly smaller file sizes than JPEG/PNG
Cache headers + CDN	CloudFront, Akamai, Fastly for global distribution
S3 for large files	Videos, CAD models → Pimcore S3 Asset Store plugin
Elasticsearch indexing	Enable for searching/browsing large asset libraries

---

Example: A Complete Product Media Setup

Product: **Laptop X1**

```
SKU: LAP123
Price: 1200
Image: laptop.jpg (Asset)
Manual: manual.pdf (Asset)
Promo Video: promo.mp4 (Asset)
```

In the frontend (CMS page or API), you can fetch and display:

- Laptop Name + Price (from Product Object)
- Product Image (from Asset — resized to thumbnail)
- Downloadable Manual PDF (from Asset)

- Promo Video embed (from Asset)

Summary

- **Assets** = the media library (files, images, videos, PDFs)
- **Metadata** = makes assets searchable and manageable
- **Thumbnails** = define reusable image transformation presets
- **Relations** = link assets to products, pages, and any object
- **Best Practice:** organized folder structure + CDN + WebP thumbnails = fast, clean media delivery

---

Chapter 5: CMS Documents

What are Documents?

Documents are the CMS layer of Pimcore. They represent web pages, reusable content blocks, and email templates. Stored in the **Documents** tree in the Admin UI, they form your website's page hierarchy.

Three Types of Documents

Type	Use Case	Example
Page	Full CMS web page	Homepage, About Us, Product Detail
Snippet	Reusable content block	Footer, Promo Banner, Newsletter Signup
Email	Email templates	Order confirmation, Marketing newsletter

---

Step 1: Create a New Page

1. In Admin Panel → **Documents** (left sidebar)

2. Right-click on root (or a folder) → **Add Document** → **Page**
3. Enter details:
   - Name: `about-us`
   - Title: "About Us"
   - Template: `default.html.twig`
4. Save & Publish

> The page is now accessible at `/about-us`

Step 2: Define Editable Areas in Twig

Pimcore's power is the **editable area** system — editors can change content directly in the Admin UI without touching code.

Example: `templates/default.html.twig`

twig

```twig
{% extends "layout.html.twig" %}

{% block content %}
    <h1>{% pimcore_input('headline', {'width': 600}) %}</h1>

    <div>
        {% pimcore_wysiwyg('content', {'height': 300}) %}
    </div>

    <div>
        {% pimcore_image('mainImage', {'thumbnail': 'content'}) %}
    </div>
{% endblock %}
```

Available Editable Types

Tag	Type	Use Case
`pimcore_input`	Single-line text	Headlines, titles
`pimcore_wysiwyg`	Rich text editor	Body content

Tag	Type	Use Case
`pimcore_image`	Image from DAM	Hero images, product photos
`pimcore_link`	Link element	CTAs, navigation links
`pimcore_snippet`	Include a Snippet	Reusable blocks (footer, banner)
`pimcore_select`	Dropdown selection	Layout options
`pimcore_areablock`	Drag-and-drop area blocks	Complex page builders

Step 3: Edit Content in Admin

1. Go to **Documents** → About Us → **Edit**
2. You'll see the WYSIWYG editor and editable fields (Headline, Content, Image)
3. Enter text in the headline field
4. Click into the WYSIWYG area to type content
5. Drag & drop an image from Assets into the image area
6. Click **Save & Publish**

> Non-technical editors can now update text & images without touching code.

Step 4: Organize Content Pages

Folder Structure Best Practice

```
/documents
  /en
    /home
    /about-us
    /products
      /laptop-x1
      /wireless-mouse
  /de
```

```
      /startseite
      /ueber-uns
      /produkte
    /fr
      /accueil
      /a-propos
```

Use Snippets for Reusable Blocks

- Create a **Snippet** document for your newsletter signup box
- Include it in any Page template using `{% pimcore_snippet('newsletter') %}`
- Update the Snippet once → all pages using it update automatically

---

Step 5: Connecting Documents to Data Objects

The real power of Pimcore CMS is combining **Documents** (pages) with **Data Objects** (structured data).

Example: Dynamic Product Detail Page

Template: `templates/product/detail.html.twig`

```
{% extends "layout.html.twig" %}

{% block content %}
    <h1>{{ product.name }}</h1>
    <p>Price: ${{ product.price }}</p>
    <div>{{ product.description|raw }}</div>

    {% if product.image %}
        <img src="{{ product.image.getThumbnail('galleryThumbnail') }}" alt="{{
product.name }}">
    {% endif %}
{% endblock %}
```

Controller Route:

```
// In Admin: Documents → product-detail page → Settings → Controller:
ProductController::detail
```
php

This way, the same template renders different product data depending on which product is loaded.

CMS Page vs. Headless Approach

Approach	When to Use
Traditional CMS (Twig templates)	Classic websites, blog, marketing pages
Headless CMS (API-first)	React/Vue/Next.js frontends, mobile apps, omnichannel delivery

Pimcore supports both. You can start with traditional CMS and add headless APIs later — without changing your data models.

Summary

- **Documents** = the CMS layer (pages, snippets, emails)
- **Editable areas** in Twig = let editors manage content without code
- **Folder structure** = organize by language and content type
- **Snippets** = reuse common blocks across pages
- **Data Object integration** = documents can load and display dynamic structured data

---

Chapter 6: Controllers & Routing

Overview

Pimcore is **100% Symfony under the hood**, so controllers work exactly like Symfony controllers. You define routes with PHP attributes or YAML, inject services via Dependency Injection, and return responses (HTML via Twig or JSON).

Project Structure

```
src/
├── Controller/
│   ├── ProductController.php       # Product list, detail, category, search
│   └── Api/
│       └── ProductApiController.php # JSON API endpoints
├── Service/
│   └── ProductService.php
└── Repository/
    └── ProductRepository.php
```

---

1. Product List Controller

File: `src/Controller/ProductController.php`

php

```php
<?php
namespace App\Controller;

use Pimcore\Model\DataObject\Product;
use Pimcore\Model\DataObject\Product\Listing;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class ProductController extends AbstractController
{
    #[Route('/products', name: 'product_list')]
    public function list(): Response
    {
        $products = new Listing();
        $products->setOrderKey('name');
        $products->setOrder('asc');

        return $this->render('product/list.html.twig', [
            'products' => $products,
        ]);
    }
}
```

Template: `templates/product/list.html.twig`

```twig
{% extends "base.html.twig" %}

{% block title %}Product Catalog{% endblock %}

{% block content %}
<h1>Product Catalog</h1>
<ul>
    {% for product in products %}
        <li>
            <a href="{{ path('product_detail', {id: product.id}) }}">
                <strong>{{ product.name }}</strong>
            </a>
            - ${{ product.price }}
        </li>
    {% else %}
        <li>No products found.</li>
    {% endfor %}
</ul>
{% endblock %}
```

---

2. Product Detail Controller

```php
#[Route('/product/{id}', name: 'product_detail')]
public function detail(int $id): Response
{
    $product = Product::getById($id);

    if (!$product) {
        throw $this->createNotFoundException("Product with ID $id not found.");
    }

    return $this->render('product/detail.html.twig', [
        'product' => $product,
        'category' => $product->getCategory(),
    ]);
}
```

Template: `templates/product/detail.html.twig`

```twig
{% extends "base.html.twig" %}

{% block content %}
<h1>{{ product.name }}</h1>
<p><strong>Price:</strong> ${{ product.price }}</p>
<div><strong>Description:</strong> {{ product.description|raw }}</div>

{% if product.image %}
    <img src="{{ product.image.getThumbnail('galleryThumbnail') }}" alt="{{
product.name }}">
{% endif %}

{% if category %}
    <hr>
    <h2>Category: {{ category.name }}</h2>
{% endif %}

<a href="{{ path('product_list') }}">← Back to catalog</a>
{% endblock %}
```

---

3. Category Filtering

```php
#[Route('/category/{id}', name: 'products_by_category')]
public function byCategory(int $id): Response
{
    $category = \Pimcore\Model\DataObject\Category::getById($id);

    if (!$category) {
        throw $this->createNotFoundException("Category not found.");
    }

    $products = new \Pimcore\Model\DataObject\Product\Listing();
    $products->setCondition('category__id = ?', [$category->getId()]);
    $products->setOrderKey('name');
    $products->setOrder('asc');

    return $this->render('product/by_category.html.twig', [
        'category' => $category,
        'products' => $products,
    ]);
}
```

4. Search Controller

```php
#[Route('/products/search', name: 'product_search')]
public function search(Request $request): Response
{
    $query = $request->query->get('q');
    $products = [];

    if ($query) {
        $products = new Listing();
        $products->setCondition('name LIKE ?', ["%$query%"]);
        $products->setOrderKey('name');
        $products->setOrder('asc');
    }

    return $this->render('product/search.html.twig', [
        'products' => $products,
        'query'    => $query,
    ]);
}
```

Template search form:

```twig
<form method="get" action="{{ path('product_search') }}">
    <input type="text" name="q" value="{{ query }}" placeholder="Search by
name">
    <button type="submit">Search</button>
</form>
```

5. Pagination

```php
#[Route('/products', name: 'product_list')]
public function list(Request $request): Response
{
    $page    = max(1, (int) $request->query->get('page', 1));
    $perPage = 10;
    $offset  = ($page - 1) * $perPage;
```

```
        $products = new Listing();
        $products->setLimit($perPage);
        $products->setOffset($offset);
        $products->setOrderKey('name');
        $products->setOrder('asc');

        $totalPages = ceil($products->getTotalCount() / $perPage);

        return $this->render('product/list.html.twig', [
            'products'   => $products,
            'page'       => $page,
            'totalPages' => $totalPages,
        ]);
    }
```

Pagination in Twig:

```twig
<div class="pagination">
    {% if page > 1 %}
        <a href="{{ path('product_list', {page: page - 1}) }}">← Previous</a>
    {% endif %}

    <span>Page {{ page }} of {{ totalPages }}</span>

    {% if page < totalPages %}
        <a href="{{ path('product_list', {page: page + 1}) }}">Next →</a>
    {% endif %}
</div>
```

---

6. Card Grid Layout (TailwindCSS)

Template: `templates/product/list.html.twig`

```twig
<div class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-6">
    {% for product in products %}
        <div class="bg-white rounded-2xl shadow-md overflow-hidden hover:shadow-lg transition">
            {% if product.image %}
                <img src="{{ product.image.getThumbnail('galleryThumbnail') }}"
                     alt="{{ product.name }}"
                     class="w-full h-48 object-cover">
```

```
            {% else %}
                <div class="w-full h-48 bg-gray-200 flex items-center justify-
center text-gray-500">
                    No Image
                </div>
            {% endif %}

            <div class="p-4">
                <h2 class="text-lg font-semibold">{{ product.name }}</h2>
                <p class="text-gray-600">${{ product.price }}</p>

                {% if product.category %}
                    <p class="text-sm text-gray-500 mt-1">
                        Category: <span class="font-medium">{{
product.category.name }}</span>
                    </p>
                {% endif %}

                <a href="{{ path('product_detail', {id: product.id}) }}"
                   class="mt-3 inline-block bg-blue-600 text-white px-3 py-1
rounded-lg text-sm hover:bg-blue-700">
                    View Details
                </a>
            </div>
        </div>
    {% endfor %}
</div>
```

---

7. Injecting a Service

php

```php
// src/Service/ProductService.php
class ProductService
{
    public function getFeaturedProducts(int $limit = 5): array
    {
        $listing = new \Pimcore\Model\DataObject\Product\Listing();
        $listing->setLimit($limit);
        $listing->setOrderKey('price');
        $listing->setOrder('DESC');
        return $listing->load();
    }
```

```
}

// In Controller:
public function featured(ProductService $productService): Response
{
    return $this->json($productService->getFeaturedProducts());
}
```

Routes Summary

URL	Route Name	Description
`/products`	`product_list`	All products (paginated)
`/product/{id}`	`product_detail`	Single product detail
`/category/{id}`	`products_by_category`	Products filtered by category
`/products/search?q=...`	`product_search`	Search by product name
`/products/sort/{field}`	`product_sort`	Sort products by field
`/api/products`	`api_products`	JSON API (see Chapter 8)

Connecting a Route to a Pimcore Document

To make a controller power a Pimcore Document page:

1. Go to **Documents** → Create Page → Name it `products`
2. Open the page → **Settings** tab → set:
   - **Controller:** `App\Controller\ProductController::list`
   - **Template:** `product/list.html.twig`
3. Publish

Now `/products` loads through both the Pimcore Document tree and your Symfony controller.

Chapter 7: REST API & GraphQL

Overview

Pimcore supports two major API approaches:

1. **Custom REST APIs** — built with Symfony controllers (full control)
2. **DataHub GraphQL** — official Pimcore bundle for flexible, frontend-friendly queries

---

Option A: Custom REST API (Symfony Controller)

Basic Product API

File: `src/Controller/Api/ProductApiController.php`

```php
<?php
namespace App\Controller\Api;

use Pimcore\Model\DataObject\Product;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\Routing\Annotation\Route;

class ProductApiController extends AbstractController
{
    #[Route('/api/products', name: 'api_products', methods: ['GET'])]
    public function list(Request $request): JsonResponse
    {
        $page  = max(1, (int) $request->query->get('page', 1));
        $limit = min(50, (int) $request->query->get('limit', 10)); // cap for safety
        $name  = $request->query->get('name');

        $listing = new Product\Listing();
        $listing->setLimit($limit);
        $listing->setOffset(($page - 1) * $limit);

        if ($name) {
```

```
            $listing->addConditionParam("name LIKE ?", ["%$name%"]);
        }

        $items = [];
        foreach ($listing as $product) {
            $items[] = [
                'id'          => $product->getId(),
                'sku'         => $product->getSku(),
                'name'        => $product->getName(),
                'price'       => $product->getPrice(),
                'description' => $product->getDescription(),
                'category'    => $product->getCategory()?->getName(),
                'image'       => $product->getImage()
                                     ? $product->getImage()->getFullPath()
                                     : null,
            ];
        }

        return $this->json([
            'page'     => $page,
            'limit'    => $limit,
            'count'    => $listing->getTotalCount(),
            'products' => $items,
        ]);
    }
}
```

Test with curl

```
# Fetch first page, 5 products
curl "http://localhost/api/products?page=1&limit=5"

# Filter by name containing 'iPhone'
curl "http://localhost/api/products?page=1&limit=10&name=iPhone"
```

Sample JSON Response

```json
{
    "page": 1,
    "limit": 5,
    "count": 27,
    "products": [
        {
```

```
            "id": 123,
            "sku": "IP16-256GB",
            "name": "iPhone 16 Pro",
            "price": 119999,
            "description": "Latest flagship phone",
            "category": "Smartphones",
            "image": "/var/assets/products/iphone16.jpg"
        }
    ]
}
```

Option B: Pimcore DataHub (GraphQL)

Installation

```
composer require pimcore/data-hub
bin/console pimcore:bundle:enable PimcoreDataHubBundle
bin/console pimcore:bundle:install PimcoreDataHubBundle
```

Create a GraphQL Endpoint

1. In Admin UI → **Settings** → **DataHub**
2. Click **Add Configuration**
3. Select **GraphQL** as type
4. Choose the Data Object class to expose (e.g., `Product`)
5. Configure which fields/relations are accessible
6. Save → Pimcore auto-generates a GraphQL endpoint

Default endpoint URL:

```
/graphql/{configName}
# Example:
http://localhost/graphql/products
```

Example GraphQL Query

```graphql
query {
  getProductListing(first: 5) {
    edges {
      node {
        id
        name
        sku
        price
        description
        images {
          edges {
            node {
              path
              thumbnails {
                product_preview {
                  path
                }
              }
            }
          }
        }
        categories {
          edges {
            node {
              id
              name
            }
          }
        }
      }
    }
  }
}
```

Fetching a Single Product

```graphql
query {
  getProduct(id: 123) {
    name
    sku
    price
    description
  }
}
```

Response:

```json
{
    "data": {
        "getProduct": {
            "name": "Laptop X1",
            "sku": "LAP123",
            "price": 1200,
            "description": "A lightweight business laptop"
        }
    }
}
```

Fetching a Document Page

```graphql
query {
  getDocument(id: 45) {
    fullPath
    title
    properties {
      key
      value
    }
  }
}
```

Comparison: REST vs GraphQL

Approach	Pros	Cons
DataHub (GraphQL)	Frontend devs love it; flexible field selection; configurable in Admin UI	Extra bundle; some setup overhead
Custom REST (Symfony)	Full control; lightweight; no extra dependencies	Fixed response shapes; less frontend flexibility
API Platform	Auto-generated REST + GraphQL APIs; powerful filters	Extra setup; can feel heavy for simple use cases

When to Use Which?

- **GraphQL (DataHub):** Frontend teams using React, Next.js, Vue that need flexible queries
- **Custom REST:** Backend-controlled APIs for performance/security
- **API Platform:** Teams that want auto-generated, standards-compliant APIs

---

Securing Your APIs

```yaml
# config/packages/security.yaml
security:
  firewalls:
    api:
      pattern: ^/api
      stateless: true
      json_login:
        check_path: /api/login
  access_control:
    - { path: ^/api, roles: ROLE_API_USER }
```

Best Practices

- Use **JWT or API keys** for machine-to-machine communication
- Never expose `/admin/*` endpoints publicly
- **Rate-limit** APIs at the proxy/load balancer level
- In DataHub: configure per-endpoint permissions (read-only, field restrictions)
- Always **cap the `limit` parameter** in REST APIs (max 50) to prevent large DB queries

---

Real-World Use Cases

Use Case	Solution
Headless website	Pimcore manages data → React/Vue fetches via GraphQL
Mobile app	iOS/Android fetches product catalog via REST

Use Case	Solution
Marketplace	Syndicate product data to Amazon, Shopify via REST
Omnichannel	Same content powers website, app, in-store displays

---

Chapter 8: Workflows & Localization

Part 1: Workflows

What are Workflows?

Workflows in Pimcore define a **series of states** that content moves through before being published. They enforce quality control and governance.

- Built on **Symfony's workflow component**
- Can be applied to Data Objects, Assets, or Documents
- Define who can do what at each stage

Example use cases:

- Product content: Draft → Review → Approved → Published
- Marketing images: Submitted → QA Review → Approved for use
- Multi-language content: Translated → Native Speaker Review → Published

---

Workflow Components

Component	Description
Places (States)	Stages of the process (Draft, Review, Approved)
Transitions	Rules to move from one state to another
Roles & Permissions	Who can execute each transition

Component	Description
UI Actions	Buttons that appear in the Pimcore Admin panel

Example: Product Approval Workflow

Goal: Product moves through: `Draft → Review → Approved → Published`

File: `var/config/workflow.yaml`

yaml

```yaml
workflows:
  product_approval:
    label: "Product Approval Workflow"
    type: 'state_machine'
    supports:
      - 'Pimcore\Model\DataObject\Product'
    places:
      - draft
      - review
      - approved
      - published
    initial_place: draft
    transitions:
      submit_for_review:
        from: draft
        to: review
      approve:
        from: review
        to: approved
      publish:
        from: approved
        to: published
      send_back:
        from: review
        to: draft
    metadata:
      submit_for_review:
        label: "Submit for Review"
      approve:
        label: "Approve"
      publish:
        label: "Publish"
```

```
        send_back:
          label: "Send Back to Draft"
```

Assign Workflow to Users

In **Settings** → **Users/Roles**, assign permissions:

Role	Allowed Transitions
Content Editor	Draft → Review (`submit_for_review`)
Manager	Review → Approved (`approve`)
Admin	Approved → Published (`publish`)

Workflow in Action

1. A new Product is created → starts in **Draft**
2. Editor clicks **Submit for Review** → moves to **Review**
3. Manager reviews content → clicks **Approve** → moves to **Approved**
4. Admin clicks **Publish** → Product goes **live**

In the Product editor screen, users see workflow action buttons based on their permissions.

Benefits of Workflows

- Ensures **quality control** before content goes live
- Provides **audit trails** (who approved what, when)
- Makes **content governance** easier for distributed teams

Part 2: Localization & Multilingual

How Pimcore Handles Localization

Component	How It Works
Data Objects	Fields can be made **localized** (per language)
Documents	Language-specific page trees (`/en` , `/de` , `/fr`)
Assets	Metadata translations (alt text, titles)
System	Configurable language list

Step 1: Configure System Languages

1. Admin → **Settings** → **System** → **Localization & Internationalization**
2. Add languages: `en` , `de` , `fr`
3. Save

Step 2: Add Localized Fields to Product Class

1. Go to Settings → Data Objects → Classes → `Product`
2. Add a **Localized Fields** container
3. Inside it, add fields like:
   - `name` (Input)
   - `description` (WYSIWYG)

Step 3: Fill Data Per Language

Open a Product (e.g., Laptop X1). You'll see language tabs (EN, DE, FR):

Language	Name	Description
EN	"Laptop X1"	"Lightweight business laptop"

Language	Name	Description
DE	"Laptop X1"	"Leichter Business-Laptop"
FR	"Laptop X1"	"Ordinateur portable léger pour entreprise"

The same product object holds all translations — no duplication needed.

Step 4: Multilingual Document Trees

```
/documents
  /en
    /about-us
    /products/laptop-x1
  /de
    /ueber-uns
    /produkte/laptop-x1
  /fr
    /a-propos
    /produits/laptop-x1
```

- Each language folder = one localized website tree
- Pimcore auto-detects language via URL prefix (`/en/` , `/de/`)

Step 5: Language Switcher in Twig

twig

```twig
<ul>
    <li>
        <a href="{{ pimcore_url(document, {'_locale': 'en'}) }}">English</a>
    </li>
    <li>
        <a href="{{ pimcore_url(document, {'_locale': 'de'}) }}">Deutsch</a>
    </li>
    <li>
        <a href="{{ pimcore_url(document, {'_locale': 'fr'}) }}">Français</a>
    </li>
</ul>
```

Step 6: Asset Metadata Translations

1. Select an image in Assets
2. Go to **Metadata** tab
3. Enter translated values per language:
   - Alt Text (EN): "Laptop X1 Product Image"
   - Alt Text (DE): "Laptop X1 Produktbild"

Localization URL Examples

URL	Language	Content
`/en/products/laptop-x1`	English	Product in English
`/de/produkte/laptop-x1`	German	Same product, German translation
`/fr/produits/laptop-x1`	French	Same product, French translation

All powered by **one Pimcore instance** with structured multilingual data — no content duplication.

Chapter 9: eCommerce & Product Catalog

What is Pimcore's eCommerce Framework?

Pimcore's eCommerce Framework is **not** a ready-made shop like Shopify. Instead, it's a **flexible toolkit** to build custom e-commerce solutions:

- Model products in Data Objects (SKU, price, variants, categories)
- Organize them into a searchable, filterable catalog
- Add pricing, availability, cart & checkout logic
- Integrate with ERP, CRM, and payment systems

Prerequisites

Before building a catalog, ensure you have:

- A `Product` class with fields: `name`, `sku`, `price`, `category`, `image`
- A `Category` class for organizing products
- At least 2–3 Product objects in the Data Objects tree

Step 1: Install the eCommerce Bundle

```
composer require pimcore/pimcore-ecommerce-framework
```

Enable in `config/bundles.php` if not auto-registered.

Step 2: Create Category Structure

1. Go to **Data Objects** → Categories
2. Add categories:
    - Electronics
    - Accessories
    - Home Appliances

Step 3: Assign Products to Categories

In the Product objects, set the `category` relation:

- Laptop X1 → Category = **Electronics**
- Wireless Mouse → Category = **Accessories**

Step 4: Configure the Product Index

Pimcore uses an **index service** (similar to Elasticsearch but also works with MySQL) to make products searchable and filterable.

File: `config/packages/pimcore_ecommerce.yaml`

```yaml
pimcore_ecommerce_framework:
  index_services:
    mysql:
      type: MySql
      config:
        tablename: ecommerce_index
  workers:
    default:
      type: default
```

Bootstrap the index:

```
bin/console ecommerce:indexservice:bootstrap
```

Step 5: Add Products to Index

In Pimcore Admin:

1. Go to **Marketing** → **E-Commerce** → **Index**
2. Select **Add Products to Index** → choose your Product objects

---

Step 6: Create a Catalog Page

1. Go to **Documents** → Add Page → name it `catalog`

Template: `templates/catalog.html.twig`

```twig
{% set products = pimcore_ecommerce_index('default').getProducts() %}

<h1>Product Catalog</h1>
<div class="product-grid">
    {% for product in products %}
        <div class="product-card">
```

```
            <h2>{{ product.name }}</h2>
            <p>SKU: {{ product.sku }}</p>
            <p>Price: {{ product.price }} €</p>
            <img src="{{ product.image|pimcore_image({'thumbnail': 'catalog'})
}}" alt="{{ product.name }}">
            <a href="{{ path('product_detail', {id: product.id}) }}">View
Details</a>
        </div>
    {% endfor %}
</div>
```

Product Relationships

Related Products (Upsells, Bundles)

Add to the `Product` class:

- `relatedProducts` → Many-to-Many Object Relation (to `Product`)
- `alternativeProducts` → Many-to-Many Object Relation (to `Product`)

Example in Admin:

- **Laptop X1** → Related Products: Wireless Mouse, Laptop Bag
- **iPhone 16** → Alternative Products: iPhone 16 Pro, iPhone 16 Plus

Display in Templates

twig

```
{% if product.relatedProducts %}
    <h3>Related Products</h3>
    {% for related in product.relatedProducts %}
        <a href="{{ path('product_detail', {id: related.id}) }}">
            {{ related.name }} – ${{ related.price }}
        </a>
    {% endfor %}
{% endif %}
```

Advanced: Filter by Category in Catalog

```php
// Controller: filter by category
#[Route('/shop/{categorySlug}', name: 'catalog_by_category')]
public function catalog(string $categorySlug): Response
{
    // Find category by slug
    $categories = new \Pimcore\Model\DataObject\Category\Listing();
    $categories->setCondition('slug = ?', [$categorySlug]);
    $category = $categories->current();

    // Fetch products in this category
    $products = new \Pimcore\Model\DataObject\Product\Listing();
    $products->setCondition('category__id = ?', [$category->getId()]);

    return $this->render('catalog.html.twig', [
        'products' => $products,
        'category' => $category,
    ]);
}
```

Cart & Checkout (Extending the Framework)

For a full e-commerce flow, you can extend with:

```php
// CartManager (Pimcore eCommerce)
$cartManager = $this->get('pimcore_ecommerce_framework.cart_manager');
$cart = $cartManager->getOrCreateCartByName('default');
$cart->addItem($product, 1);
$cart->save();
```

The framework provides:

- **CartManager** — manage sessions/persistent carts
- **PriceSystem** — flexible pricing rules (customer groups, promotions)
- **AvailabilitySystem** — stock management
- **Checkout** — multi-step order processing

Summary

Feature	How to Implement
Product listing	`Product\Listing()` + controller
Category filtering	`setCondition('category__id = ?')`
Product search	`setCondition('name LIKE ?')`
Product index	`pimcore_ecommerce_framework` config + bootstrap
Related products	Many-to-Many relation in class definition
Cart	`CartManager` from eCommerce Framework
Checkout	Custom Symfony controllers + CartManager
Integrations	ERP/CRM via Symfony services / events

> **Next Steps:** Add faceted filtering (category, price range) using the Index Service, or integrate with a payment gateway via Symfony event subscribers.

Chapter 10: Bundles, Plugins & Extensions

Bundles vs. Plugins

	Bundles (Symfony)	Plugins (Pimcore)
What they are	Standard Symfony packages	Pimcore-specific UI/feature extensions
Installed via	Composer	Composer or manually under `/bundles`
Can add	Services, controllers, templates, commands	Custom admin menus, dashboard widgets, object field types, workflows

	Bundles (Symfony)	Plugins (Pimcore)
Use when	Business logic, APIs, integrations	Extending admin UI, editor features

---

Building a Custom Pimcore Bundle

We'll build a **DemoBundle** as a complete example.

Bundle Directory Layout

```
project-root/
└── bundles/
    └── DemoBundle/
        ├── composer.json
        ├── DemoBundle.php
        └── src/
            ├── Controller/
            │   └── DefaultController.php
            ├── Service/
            │   └── GreetingService.php
            └── Resources/
                └── config/
                    └── services.yaml
```

> **Convention:** Custom bundles live in `/bundles/` to stay isolated from app logic in `/src/`.

Step 1: composer.json (for the bundle)

json

```
{
    "name": "mycompany/demo-bundle",
    "type": "pimcore-bundle",
    "description": "A demo Pimcore bundle",
    "autoload": {
        "psr-4": {
            "DemoBundle\\": "src/"
        }
```

```
    },
    "require": {
        "php": ">=8.1"
    }
}
```

Step 2: The Bundle Class

php

```php
<?php
// bundles/DemoBundle/DemoBundle.php
namespace DemoBundle;

use Pimcore\Extension\Bundle\AbstractPimcoreBundle;

class DemoBundle extends AbstractPimcoreBundle
{
    public function getJsPaths(): array
    {
        // Register custom admin UI JavaScript files
        return [
            '/bundles/demobundle/js/admin.js',
        ];
    }

    public function getCssPaths(): array
    {
        return [];
    }
}
```

Step 3: A Service

php

```php
<?php
// bundles/DemoBundle/src/Service/GreetingService.php
namespace DemoBundle\Service;

class GreetingService
{
    public function greet(string $name): string
```

```
    {
        return "Hello, " . $name . "!";
    }
}
```

Step 4: A Controller

```php
<?php
// bundles/DemoBundle/src/Controller/DefaultController.php
namespace DemoBundle\Controller;

use DemoBundle\Service\GreetingService;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class DefaultController extends AbstractController
{
    #[Route('/demo/hello/{name}', name: 'demo_hello')]
    public function hello(string $name, GreetingService $greeter): Response
    {
        return $this->json([
            'message' => $greeter->greet($name)
        ]);
    }
}
```

Visiting `/demo/hello/Sandeep` returns: `{ "message": "Hello, Sandeep!" }`

Step 5: services.yaml

```yaml
# bundles/DemoBundle/src/Resources/config/services.yaml
services:
    DemoBundle\:
        resource: '../../*'
        exclude: '../../{Resources}'
```

Step 6: Enable the Bundle

```
bin/console pimcore:bundle:enable DemoBundle
bin/console pimcore:bundle:install DemoBundle

# Verify
bin/console debug:bundle
```

Extending the Admin UI (ExtJS)

Pimcore's admin is built with **ExtJS**. You can add custom menus, toolbar buttons, and panels.

Add a Toolbar Button

javascript

```
// bundles/DemoBundle/src/Resources/public/js/admin.js

pimcore.registerNS("pimcore.plugin.DemoBundle");

pimcore.plugin.DemoBundle = Class.create(pimcore.plugin.admin, {
    getClassName: function () {
        return "pimcore.plugin.DemoBundle";
    },

    initialize: function () {
        pimcore.plugin.broker.registerPlugin(this);
    },

    pimcoreReady: function (params, broker) {
        var toolbar = pimcore.globalmanager.get("layout_toolbar");

        toolbar.add({
            text: "Demo Page",
            iconCls: "pimcore_icon_overlay_add",
            handler: function () {
                Ext.Ajax.request({
                    url: "/admin/demo/hello",
                    success: function (response) {
                        var data = Ext.decode(response.responseText);
                        Ext.Msg.alert("Demo Bundle", data.message);
                    }
```

```
                });
            }
        });
    }
});

new pimcore.plugin.DemoBundle();
```

Add a Button to the Product Grid

```javascript
// Inside admin.js — extend existing object grids
pimcore.object.grid.prototype.getToolbar =
pimcore.object.grid.prototype.getToolbar.createSequence(function () {
    if (this.class === "Product") {
        this.toolbar.add({
            text: "Custom Action",
            iconCls: "pimcore_icon_apply",
            handler: function () {
                var selected = this.getSelectionModel().getSelection();
                if (selected.length) {
                    Ext.Msg.alert("Custom Action", "Selected product ID: " +
selected[0].data.id);
                }
            }.bind(this)
        });
    }
});
```

Admin Backend Controller for Custom Pages

```php
<?php
// bundles/DemoBundle/src/Controller/AdminController.php
namespace DemoBundle\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\Routing\Annotation\Route;

#[Route('/admin/demo', name: 'demo_admin_')]
class AdminController extends AbstractController
```

```
{
    #[Route('/hello', name: 'hello')]
    public function hello(): JsonResponse
    {
        return new JsonResponse(['message' => 'Hello from Demo Admin
Controller!']);
    }
}
```

How It All Fits Together

```
1. DemoBundle.php  →  getJsPaths() registers admin.js
2. admin.js        →  hooks into Pimcore UI lifecycle via pimcore.plugin.broker
3. AdminController →  provides JSON APIs for AJAX calls from admin.js
4. Result          →  editors see new UI buttons and panels seamlessly
integrated
```

Custom Package in a Pimcore Project (Alternative Layout)

If you want to put a package inside `src/` :

```
/src/MyPackage/
├── Controller/
│   └── AdminController.php
├── Resources/
│   └── config/
│       └── routes.yaml
└── MyPackageBundle.php
```

Register routes in `config/routes.yaml` :

yaml

```
my_package:
    resource: '../src/MyPackage/Resources/config/routes.yaml'
```

Register as service in `config/services.yaml` :

```yaml
services:
    App\MyPackage\:
        resource: '../src/MyPackage/*'
        tags: ['controller.service_arguments']
```

Pro Tips

- **Share publicly:** Push to GitHub/Packagist → install via `composer require mycompany/demo-bundle`
- **Bundles = Symfony power** (custom logic, APIs, integrations)
- **Plugins = Deep Pimcore integration** (UI, workflows, DAM, PIM features)
- Always run `bin/console cache:clear` after enabling/installing bundles

Chapter 11: Event Subscribers & Hooks

Overview

Pimcore builds on Symfony's event system and adds its own events for Data Objects, Assets, and Documents. Event subscribers let you **react to things happening** in Pimcore without modifying core code.

Common uses:

- Auto-generate a slug when a product is saved
- Send a notification when content is published
- Sync data to external systems after save
- Log admin actions for auditing

Pimcore Data Object Events

```php
use Pimcore\Event\DataObjectEvents;
```

```
DataObjectEvents::PRE_UPDATE   // before saving an existing object
DataObjectEvents::POST_UPDATE  // after saving
DataObjectEvents::PRE_ADD      // before creating a new object
DataObjectEvents::POST_ADD     // after creating
DataObjectEvents::PRE_DELETE   // before deleting
DataObjectEvents::POST_DELETE  // after deleting
```

Example 1: Auto-Generate Slug on Product Save

Step 1: Add `slug` Field to Product Class

In Admin UI → Settings → Data Objects → Classes → Product:

- Add a field: `slug` (type: Input, optional: read-only)

Step 2: Create the Event Subscriber

File: `src/EventSubscriber/ProductSlugSubscriber.php`

php

```php
<?php
namespace App\EventSubscriber;

use Pimcore\Event\DataObjectEvents;
use Pimcore\Event\Model\DataObjectEvent;
use Symfony\Component\EventDispatcher\EventSubscriberInterface;
use Pimcore\Model\DataObject\Product;

class ProductSlugSubscriber implements EventSubscriberInterface
{
    public static function getSubscribedEvents(): array
    {
        return [
            DataObjectEvents::PRE_UPDATE => 'onProductSave',
            DataObjectEvents::PRE_ADD    => 'onProductSave',
        ];
    }

    public function onProductSave(DataObjectEvent $event): void
    {
        $object = $event->getObject();
```

```
        // Only handle Product objects
        if (!$object instanceof Product) {
            return;
        }

        $name = $object->getName();
        if ($name) {
            // Slugify: lowercase + replace non-alphanumeric with dashes
            $slug = strtolower(trim(preg_replace('/[^A-Za-z0-9-]+/', '-',
$name), '-'));
            $object->setSlug($slug);
        }
    }
}
```

Step 3: Register in services.yaml

```yaml
# config/services.yaml
services:
    App\EventSubscriber\ProductSlugSubscriber:
        tags:
            - { name: kernel.event_subscriber }
```

What Happens

1. You save or create a Product: "iPhone 16 Pro"
2. Event fires → `onProductSave()` runs
3. Slug is auto-populated: `iphone-16-pro`
4. Object is persisted with the slug already set

Example 2: Admin Audit Log (Login Tracking)

File: `src/EventSubscriber/AdminAuditSubscriber.php`

```php
<?php
namespace App\EventSubscriber;

use Pimcore\Event\AdminEvents;
```

```
use Symfony\Component\EventDispatcher\EventSubscriberInterface;

class AdminAuditSubscriber implements EventSubscriberInterface
{
    public static function getSubscribedEvents(): array
    {
        return [
            AdminEvents::ADMIN_LOGIN => 'onAdminLogin',
        ];
    }

    public function onAdminLogin($event): void
    {
        $request = $event->getRequest();
        $user = $event->getUser();

        \Pimcore\Log\Simple::log(
            'admin_audit',
            "User {$user->getName()} logged in from {$request->getClientIp()}"
        );
    }
}
```

Example 3: Sync to External System After Product Save

php

```
<?php
namespace App\EventSubscriber;

use Pimcore\Event\DataObjectEvents;
use Pimcore\Event\Model\DataObjectEvent;
use Symfony\Component\EventDispatcher\EventSubscriberInterface;
use Pimcore\Model\DataObject\Product;
use App\Service\ExternalSyncService;

class ProductSyncSubscriber implements EventSubscriberInterface
{
    public function __construct(private ExternalSyncService $syncService)
    {}

    public static function getSubscribedEvents(): array
    {
        return [
```

```
            DataObjectEvents::POST_UPDATE => 'onProductSaved',
        ];
    }

    public function onProductSaved(DataObjectEvent $event): void
    {
        $object = $event->getObject();

        if (!$object instanceof Product) {
            return;
        }

        // Sync to ERP, Elasticsearch, external API, etc.
        $this->syncService->syncProduct($object);
    }
}
```

Available Events

Data Object Events

Event	When
`PRE_ADD`	Before a new object is created
`POST_ADD`	After a new object is created
`PRE_UPDATE`	Before an existing object is saved
`POST_UPDATE`	After an existing object is saved
`PRE_DELETE`	Before an object is deleted
`POST_DELETE`	After an object is deleted
`PRE_PUBLISH`	Before an object is published
`POST_PUBLISH`	After an object is published

Asset Events

```php
use Pimcore\Event\AssetEvents;

AssetEvents::PRE_UPDATE
AssetEvents::POST_UPDATE
AssetEvents::PRE_ADD
AssetEvents::POST_ADD
```

Document Events

```php
use Pimcore\Event\DocumentEvents;

DocumentEvents::PRE_UPDATE
DocumentEvents::POST_UPDATE
```

Admin Events

```php
use Pimcore\Event\AdminEvents;

AdminEvents::ADMIN_LOGIN
AdminEvents::ADMIN_LOGOUT
```

Tips & Advanced Usage

Unique Slugs

If you need unique slugs, check for existing ones and append a counter:

```php
$baseSlug = $slug;
$counter = 1;
while ($this->slugExists($slug, $object->getId())) {
    $slug = $baseSlug . '-' . $counter++;
}
$object->setSlug($slug);
```

Use Symfony's SluggerInterface

```php
use Symfony\Component\String\Slugger\SluggerInterface;

class ProductSlugSubscriber implements EventSubscriberInterface
{
    public function __construct(private SluggerInterface $slugger)
    {}

    public function onProductSave(DataObjectEvent $event): void
    {
        $object = $event->getObject();
        if (!$object instanceof Product) return;

        $slug = strtolower($this->slugger->slug($object->getName()));
        $object->setSlug($slug);
    }
}
```

preUpdate vs postUpdate

Use	When
`PRE_UPDATE`	Modify the object **before** it's saved (slug generation, validation)
`POST_UPDATE`	Act **after** save (sync to Elasticsearch, send webhook, update cache)

> Never save the object again inside a `PRE_UPDATE` subscriber — it causes infinite loops.

Chapter 12: Data Import & Export

Overview

Pimcore projects frequently need to import data from CSVs, XMLs, or external APIs, and export data for syndication, backups, or reporting. This chapter covers a complete import pipeline with error handling.

CSV Import: Step by Step

Example CSV Format

```csv
sku,name,price,description,image_url
P001,Phone X,999,Latest smartphone,https://example.com/images/phone.jpg
P002,Laptop Z,1499,High-end laptop,https://example.com/images/laptop.jpg
P003,Wireless Mouse,29,Ergonomic mouse,https://example.com/images/mouse.jpg
```

Full Import Command

File: `src/Command/ImportProductsCommand.php`

```php
<?php
namespace App\Command;

use Pimcore\Model\DataObject\Product;
use Pimcore\Model\Asset;
use Pimcore\Model\Asset\Image;
use Symfony\Component\Console\Command\Command;
use Symfony\Component\Console\Input\InputInterface;
use Symfony\Component\Console\Output\OutputInterface;
use Symfony\Component\Console\Style\SymfonyStyle;

class ImportProductsCommand extends Command
{
    protected static $defaultName = 'app:import-products';

    protected function execute(InputInterface $input, OutputInterface $output):
int
    {
        $io = new SymfonyStyle($input, $output);

        $csvPath = PIMCORE_PROJECT_ROOT . '/var/import/products.csv';

        if (!file_exists($csvPath)) {
            $io->error("CSV not found: $csvPath");
            return Command::FAILURE;
        }
```

```
        $handle = fopen($csvPath, 'r');
        $header = fgetcsv($handle, 1000, ','); // first row = headers
        $count = 0;

        while (($row = fgetcsv($handle, 1000, ',')) !== false) {
            $data = array_combine($header, $row);

            try {
                $sku = $data['sku'] ?? null;
                if (!$sku) {
                    throw new \Exception("Missing SKU in row");
                }

                // Find existing product by SKU or create new
                $product = Product::getBySku($sku, 1);
                if (!$product) {
                    $product = new Product();
                    $product->setKey(\Pimcore\File::getValidFilename($sku));
                    $product->setParentId(1); // root folder
                    $product->setPublished(true);
                    $product->setSku($sku);
                }

                $product->setName($data['name'] ?? '');
                $product->setPrice((float)($data['price'] ?? 0));
                $product->setDescription($data['description'] ?? '');

                // Handle image download
                if (!empty($data['image_url'])) {
                    $asset = $this->downloadImage($data['image_url'], $sku);
                    if ($asset instanceof Image) {
                        $product->setImages([$asset]);
                    }
                }

                $product->save();
                $count++;
                $io->success("Imported/Updated: {$sku}");

            } catch (\Throwable $e) {
                $io->error("Error importing row [{$sku}]: " . $e-
>getMessage());
            }
        }

        fclose($handle);
```

```
            $io->success("Import finished. Total products: $count");

            return Command::SUCCESS;
        }

        private function downloadImage(string $url, string $sku): ?Image
        {
            $contents = @file_get_contents($url);
            if ($contents === false) {
                throw new \Exception("Failed to download image from $url");
            }

            $filename = $sku . '-' . basename(parse_url($url, PHP_URL_PATH));

            // Get or create the /imported folder
            $folder = Asset::getByPath('/imported');
            if (!$folder) {
                $folder = new Asset\Folder();
                $folder->setKey('imported');
                $folder->setParentId(1);
                $folder->save();
            }

            // Return existing if already imported
            $existing = Asset::getByPath('/imported/' . $filename);
            if ($existing instanceof Image) {
                return $existing;
            }

            // Create new asset
            $image = new Image();
            $image->setParent($folder);
            $image->setFilename($filename);
            $image->setData($contents);
            $image->save();

            return $image;
        }
    }
```

Run the Import

```
    # Place your CSV file
    mkdir -p var/import
```

```
cp your-products.csv var/import/products.csv

# Run the import command
bin/console app:import-products
```

Handling Categories from CSV

Extend the CSV format:

```csv
sku,name,price,description,category_path,image_url
P001,Phone X,999,Latest smartphone,/categories/electronics,https://...
```

Add to the import:

```php
// Find/create category
if (!empty($data['category_path'])) {
    $category =
\Pimcore\Model\DataObject\Category::getByPath($data['category_path']);
    if ($category) {
        $product->setCategory($category);
    }
}
```

Export Products to CSV

File: `src/Command/ExportProductsCommand.php`

```php
<?php
namespace App\Command;

use Pimcore\Model\DataObject\Product\Listing;
use Symfony\Component\Console\Command\Command;
use Symfony\Component\Console\Input\InputInterface;
use Symfony\Component\Console\Output\OutputInterface;
use Symfony\Component\Console\Style\SymfonyStyle;

class ExportProductsCommand extends Command
```

```
{
    protected static $defaultName = 'app:export-products';

    protected function execute(InputInterface $input, OutputInterface $output):
int
    {
        $io = new SymfonyStyle($input, $output);
        $exportPath = PIMCORE_PROJECT_ROOT . '/var/export/products_export.csv';

        // Ensure export directory exists
        if (!is_dir(dirname($exportPath))) {
            mkdir(dirname($exportPath), 0755, true);
        }

        $handle = fopen($exportPath, 'w');
        fputcsv($handle, ['id', 'sku', 'name', 'price', 'description',
'category']);

        $listing = new Listing();
        $listing->setOrderKey('name');
        $listing->setOrder('asc');

        foreach ($listing as $product) {
            fputcsv($handle, [
                $product->getId(),
                $product->getSku(),
                $product->getName(),
                $product->getPrice(),
                strip_tags($product->getDescription()),
                $product->getCategory()?->getName() ?? '',
            ]);
        }

        fclose($handle);

        $io->success("Exported to: $exportPath");
        return Command::SUCCESS;
    }
}


bin/console app:export-products
```

Using League\CSV for Better CSV Handling

For large files or complex CSV requirements:

```
composer require league/csv
```

```php
use League\Csv\Reader;
use League\Csv\Writer;

// Reading
$reader = Reader::createFromPath('/var/import/products.csv', 'r');
$reader->setHeaderOffset(0);

foreach ($reader->getRecords() as $record) {
    // $record['sku'], $record['name'], etc.
}

// Writing
$writer = Writer::createFromPath('/var/export/products.csv', 'w');
$writer->insertOne(['sku', 'name', 'price']);
$writer->insertAll($rows);
```

Import via Pimcore Admin UI (Built-in)

Pimcore also has a **CSV Import** feature built into the Admin:

1. Go to **Data Objects** → select a class
2. Click **Import CSV** button in the toolbar
3. Map CSV columns to class fields
4. Preview & execute import

This is useful for non-technical users but has less flexibility than a custom command.

Error Handling Best Practices

Strategy	Implementation
Per-row try/catch	Don't let one bad row stop the entire import
Log errors to file	`\Pimcore\Log\Simple::log('import', $message)`
Dry-run mode	Add `--dry-run` flag to simulate import without saving
SKU deduplication	Always check `getBySku()` before creating new objects
Image download retry	Wrap HTTP calls in retry logic for flaky URLs

Adding a Dry-Run Flag

```php
protected function configure(): void
{
    $this->addOption('dry-run', null, InputOption::VALUE_NONE, 'Simulate
without saving');
}

// In execute():
$dryRun = $input->getOption('dry-run');
if (!$dryRun) {
    $product->save();
}


bin/console app:import-products --dry-run
```

Chapter 13: Performance & Caching

Overview

Pimcore is powerful but can get heavy if not tuned right. Performance in Pimcore comes from stacking multiple cache layers, tuning the database, using search indexing for large datasets, and profiling to find bottlenecks.

Cache Layer 1: PHP OPcache

PHP OPcache compiles PHP files to bytecode and keeps them in memory — essential for production.

Config in `php.ini`:

```ini
opcache.enable=1
opcache.memory_consumption=256
opcache.max_accelerated_files=20000
opcache.validate_timestamps=0   ; only set to 0 in production
opcache.revalidate_freq=0       ; production
opcache.preload=/app/var/cache/opcache_preload.php  ; optional preloading
```

Cache Layer 2: Pimcore/Symfony Cache (Redis)

Pimcore uses Symfony Cache for metadata, class definitions, and object bricks. The default filesystem adapter is slow — **switch to Redis** in production.

Install Redis extension:

```
sudo apt install redis-server php-redis
```

Config: `config/packages/cache.yaml`

```yaml
framework:
  cache:
    app: cache.adapter.redis
    default_redis_provider: "redis://localhost:6379"
    pools:
      pimcore.cache.pool:
        adapter: cache.adapter.redis
        provider: "redis://localhost:6379"
```

Verify Redis is running:

```
redis-cli ping
# Expected: PONG
```

Cache Layer 3: Full-Page Cache (Varnish / Nginx)

For CMS pages and API responses that don't change often, full-page caching prevents requests from hitting PHP at all.

Nginx FastCGI Cache Setup

```nginx
# /etc/nginx/nginx.conf
fastcgi_cache_path /tmp/nginx_cache levels=1:2 keys_zone=PIMCORE:100m
inactive=60m;
fastcgi_cache_key "$scheme$request_method$host$request_uri";

server {
    location ~ \.php$ {
        fastcgi_cache PIMCORE;
        fastcgi_cache_valid 200 60m;
        fastcgi_cache_use_stale error timeout updating http_500;
        add_header X-Cache-Status $upstream_cache_status;
    }
}
```

Pimcore Cache Headers in Twig

```twig
{% do pimcore_cache(3600) %}  {# Cache this page for 1 hour #}
```

Database Tuning (MySQL / MariaDB)

Key Settings in `my.cnf`

```ini
[mysqld]
innodb_buffer_pool_size = 4G          ; 60-70% of available RAM
innodb_log_file_size = 256M
innodb_flush_log_at_trx_commit = 2    ; better write performance (lose max 1s
on crash)
query_cache_type = 0                  ; disable in MySQL 8.x (deprecated)
max_connections = 200
```

Add Indexes on Frequently Filtered Fields

```sql
-- Example: if you frequently filter by SKU or category
ALTER TABLE object_query_1 ADD INDEX idx_sku (sku);
ALTER TABLE object_query_1 ADD INDEX idx_category_id (category__id);
```

Avoid Expensive Queries

```php
// BAD: loads all products with all relations eagerly
$listing = new Product\Listing();
$all = $listing->load();

// GOOD: paginate and only load what you need
$listing = new Product\Listing();
$listing->setLimit(20);
$listing->setOffset(0);
$listing->setOrderKey('name');
// Only call getRelations() when actually needed
```

Enable MySQL Slow Query Log

```ini
[mysqld]
slow_query_log = 1
slow_query_log_file = /var/log/mysql/slow.log
long_query_time = 1   ; log queries taking > 1 second
```

Elasticsearch / OpenSearch for Large Catalogs

For catalogs with more than ~10,000 products, MySQL fulltext search becomes slow. Use **Elasticsearch** for fast, faceted product search.

Flow

```
Product saved in Pimcore
        ↓
```

```
Event subscriber fires (POST_UPDATE)
        ↓
Push document to Elasticsearch
        ↓
Frontend queries ES directly (fast)
```

Example ES Document

```json
{
    "sku": "P001",
    "name": "Phone X",
    "price": 999,
    "categories": ["Smartphones", "Electronics"],
    "attributes": {
        "battery": "4000mAh",
        "display": "6.5 inch OLED"
    }
}
```

Sync on Save (Event Subscriber)

```php
class ProductSearchIndexSubscriber implements EventSubscriberInterface
{
    public function __construct(private ElasticsearchClient $esClient) {}

    public static function getSubscribedEvents(): array
    {
        return [DataObjectEvents::POST_UPDATE => 'indexProduct'];
    }

    public function indexProduct(DataObjectEvent $event): void
    {
        $product = $event->getObject();
        if (!$product instanceof Product) return;

        $this->esClient->index([
            'index' => 'products',
            'id'    => $product->getId(),
            'body'  => [
                'sku'        => $product->getSku(),
                'name'       => $product->getName(),
                'price'      => $product->getPrice(),
                'category'   => $product->getCategory()?->getName(),
```

```
            ]
        ]);
    }
}
```

Profiling & Identifying Bottlenecks

1. Symfony Profiler (Dev Mode)

The Symfony debug toolbar appears at the bottom of every page in dev mode.

Click it → see:

- Timeline (controller, Twig, events)
- All SQL queries with execution time
- Cache hits/misses
- Logs emitted during the request

Enable if missing in `config/packages/dev/web_profiler.yaml` :

```yaml
web_profiler:
  toolbar: true
  intercept_redirects: false
```

2. Blackfire.io (Production Profiling)

Blackfire generates flame graphs showing exactly where PHP time is spent.

```
# Install Blackfire agent
curl https://packages.blackfire.io/gpg.key | sudo apt-key add -
echo "deb http://packages.blackfire.io/debian any main" | sudo tee
/etc/apt/sources.list.d/blackfire.list
sudo apt update && sudo apt install blackfire-php

# Profile a request
blackfire curl http://localhost/products
```

Performance Checklist

Item	Status
Enable OPcache in production	✅
Use Redis for Symfony/Pimcore cache	✅
Configure Nginx/Varnish full-page cache	✅
Tune MySQL buffer pool (60-70% RAM)	✅
Add DB indexes on filtered fields	✅
Enable MySQL slow query log	✅
Use Elasticsearch for large catalogs (>10k products)	✅
Profile with Symfony Profiler + Blackfire	✅
Paginate all listings (never load ALL records)	✅
Use lazy loading for relations	✅
Serve images via thumbnails + CDN	✅

Quick Command Reference

```
# Clear all caches
bin/console cache:clear
bin/console pimcore:cache:clear --all

# Warm up caches
bin/console cache:warmup --env=prod

# Check system requirements
bin/console pimcore:system:requirements:check

# Test database query
bin/console doctrine:query:sql "SELECT COUNT(*) FROM objects WHERE
o_type='object'"
```

```
# Check Elasticsearch connection
curl http://localhost:9200/_cluster/health
```

Chapter 14: Security & Permissions

Overview

Pimcore has a granular permission model because editors, developers, and API consumers all interact with the system differently. Security is handled at multiple layers:

1. **Users & Roles** — who can access the admin
2. **Object/Document/Asset permissions** — what content they can touch
3. **API security** — Symfony firewall + JWT/API keys
4. **Auditing** — log who did what

---

1. Users & Roles

- **Users** = actual accounts (human editors or system/API accounts)
- **Roles** = reusable permission sets assigned to multiple users
- Users can belong to **multiple roles** — permissions are merged (most permissive wins)

Best Practice: Define roles first, then assign users to roles.

Example roles:

- `content_editor` — can create/edit CMS pages
- `product_manager` — can manage Products and Categories
- `api_client` — API access only (no admin UI)

Admin UI: Settings → Users / Roles

---

2. Object / Document / Asset Permissions

Pimcore's tree-based structure makes it natural to apply permissions hierarchically.

Available Permission Types

Permission	Description
List	Can see the element in the tree
View	Can open and read the element
Create	Can create new elements
Delete	Can delete elements
Publish	Can publish/unpublish
Write	Can edit field values

Applying Permissions (Best Practice)

1. Create a folder (e.g., `/Products`, `/MarketingDocs`)
2. Right-click the folder → **Security** tab
3. Add the role (e.g., `product_manager`)
4. Grant permissions (List, View, Create, Delete, Publish, Write)
5. Children **inherit** from the parent folder

> Keep the `admin` role only for superusers. Give editors the **minimum necessary permissions** (least privilege principle).

Example Permission Structure

```
/data-objects
  /products          → product_manager: Read + Write + Publish
    /electronics     → (inherits from /products)
    /clothing        → (inherits from /products)
  /customers         → customer_manager: Read only (GDPR)
  /orders            → order_manager: Read + Write

/assets
```

```
    /product-images   → product_manager: Upload + Edit
    /legal-docs       → legal_team only

  /documents
    /en               → content_editor: Create + Edit + Publish
    /de               → german_editor: Create + Edit + Publish
```

---

3. Securing APIs

Symfony Firewall Configuration

yaml

```yaml
# config/packages/security.yaml
security:
  providers:
    pimcore_admin:
      id: Pimcore\Security\User\UserProvider

  firewalls:
    # Protect admin UI
    pimcore_admin:
      pattern: ^/admin
      anonymous: false
      provider: pimcore_admin

    # API endpoints — stateless with token auth
    api:
      pattern: ^/api
      stateless: true
      json_login:
        check_path: /api/login
        username_path: username
        password_path: password
      jwt: true  # if using lexik/jwt-authentication-bundle

  access_control:
    - { path: ^/admin, roles: ROLE_PIMCORE_ADMIN }
    - { path: ^/api,   roles: ROLE_API_USER }
```

JWT Authentication Setup

```
composer require lexik/jwt-authentication-bundle
```

```yaml
# config/packages/lexik_jwt_authentication.yaml
lexik_jwt_authentication:
    secret_key: '%env(JWT_SECRET_KEY)%'
    public_key: '%env(JWT_PUBLIC_KEY)%'
    pass_phrase: '%env(JWT_PASSPHRASE)%'
    token_ttl: 3600  # 1 hour
```

```
# Generate keys
php bin/console lexik:jwt:generate-keypair
```

Get a token:

```
curl -X POST http://localhost/api/login \
  -H "Content-Type: application/json" \
  -d '{"username": "api_user", "password": "secret"}'
```

Use the token:

```
curl http://localhost/api/products \
  -H "Authorization: Bearer eyJ0eXAiOiJ..."
```

---

4. DataHub (GraphQL) Security

```yaml
# In DataHub configuration (Admin UI):
# - Assign permissions per endpoint
# - Example: Product API allows only queries (no mutations)
# - Control which fields are exposed (hide sensitive fields)
# - Set allowed IP ranges for API access
```

---

5. Auditing Admin Actions

Built-in Logs

Pimcore logs user actions in `var/log/admin-YYYY-MM-DD.log`:

```
2025-09-26 12:01:23 User "editor1" published object ID 123
2025-09-26 12:05:44 User "manager1" deleted asset ID 456
```

Versioning

Enable **versioning** on Data Objects → full history of changes is stored:

1. Open Settings → Data Objects → Classes → `Product`
2. Enable "Save Versions" in class settings

Custom Audit Subscriber

php

```php
<?php
namespace App\EventSubscriber;

use Pimcore\Event\DataObjectEvents;
use Pimcore\Event\Model\DataObjectEvent;
use Symfony\Component\EventDispatcher\EventSubscriberInterface;
use Pimcore\Model\DataObject\Product;

class ProductAuditSubscriber implements EventSubscriberInterface
{
    public static function getSubscribedEvents(): array
    {
        return [
            DataObjectEvents::POST_DELETE => 'onProductDeleted',
            DataObjectEvents::POST_UPDATE => 'onProductUpdated',
        ];
    }

    public function onProductDeleted(DataObjectEvent $event): void
    {
        $product = $event->getObject();
        if (!$product instanceof Product) return;

        \Pimcore\Log\Simple::log(
            'audit',
            sprintf("Product deleted: ID=%d, SKU=%s", $product->getId(),
$product->getSku())
        );
    }
```

```
    public function onProductUpdated(DataObjectEvent $event): void
    {
        $product = $event->getObject();
        if (!$product instanceof Product) return;

        \Pimcore\Log\Simple::log(
            'audit',
            sprintf("Product updated: ID=%d, Name=%s", $product->getId(),
$product->getName())
        );
    }
}
```

---

6. API Key Authentication (Simpler Alternative to JWT)

For server-to-server or simple API clients:

php

```
// src/EventSubscriber/ApiKeyAuthenticator.php
namespace App\Security;

use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\Security\Core\Exception\AuthenticationException;
use Symfony\Component\Security\Http\Authenticator\AbstractAuthenticator;
use Symfony\Component\Security\Http\Authenticator\Passport\Passport;
use
Symfony\Component\Security\Http\Authenticator\Passport\SelfValidatingPassport;
use Symfony\Component\Security\Http\Authenticator\Passport\Badge\UserBadge;

class ApiKeyAuthenticator extends AbstractAuthenticator
{
    public function supports(Request $request): ?bool
    {
        return $request->headers->has('X-API-KEY');
    }

    public function authenticate(Request $request): Passport
    {
        $apiKey = $request->headers->get('X-API-KEY');
        return new SelfValidatingPassport(new UserBadge($apiKey));
    }
```

```
    public function onAuthenticationFailure(Request $request,
AuthenticationException $exception): JsonResponse
    {
        return new JsonResponse(['error' => 'Invalid API Key'], 401);
    }

    public function onAuthenticationSuccess(Request $request, $token, string
$firewallName): ?Response
    {
        return null; // continue request
    }
}
```

Security Best Practices Checklist

Practice	Description
Use roles, not per-user permissions	Easier to manage at scale
Least privilege	Give only what's needed
Secure folder trees	Apply at folder root, let children inherit
Stateless APIs	Use JWT or API keys — never session-based
Never expose `/admin` publicly	Use IP whitelisting or VPN
Rate-limit APIs	At proxy/load balancer level
Enable versioning	Full change history for Data Objects
Audit logs	Keep `admin-*.log` files and custom event log
Separate system accounts	API clients get their own user (not human editor accounts)
Review DataHub configs	Only expose fields that are safe for public consumption

Chapter 15: Debugging & Troubleshooting

Overview

Pimcore is Symfony under the hood, so you use Symfony tools plus Pimcore-specific logs. The key tools are: **log files**, **Symfony profiler**, **SQL debugging**, and a set of **console commands** for common problems.

---

1. Log Files

Pimcore writes all logs to `var/log/` (per environment).

Log File	Contents
`var/log/dev.log`	All runtime events in dev mode
`var/log/prod.log`	Runtime events in prod (less verbose)
`var/log/system.log`	Pimcore core system events
`var/log/admin-YYYY-MM-DD.log`	Admin actions (who saved/published what)
`var/log/php_error.log`	PHP errors (depending on PHP config)

Tail logs in real-time

```
# Watch dev log
tail -f var/log/dev.log

# Watch all logs
tail -f var/log/*.log

# Watch logs inside Docker
docker compose logs -f php
```

Configure Log Level

```yaml
# config/packages/monolog.yaml
monolog:
  handlers:
    main:
      type: stream
      path: "%kernel.logs_dir%/%kernel.environment%.log"
      level: debug   # use 'info' in prod, 'debug' in dev
```

---

2. Symfony Profiler (Dev Mode)

The Symfony debug toolbar appears at the **bottom of every page** in dev mode.

Enable if Missing

```yaml
# config/packages/dev/web_profiler.yaml
web_profiler:
  toolbar: true
  intercept_redirects: false
```

What You Can See

- **Timeline** — controller execution, Twig rendering, event dispatching
- **SQL queries** — each query with time + EXPLAIN link
- **Cache** — hits and misses
- **Logs** — all log messages during the request
- **Security** — current user, roles, firewall

URL: `http://localhost/_profiler`

---

3. Debugging SQL Queries

Enable Doctrine Query Logging

```
# In .env.local
APP_DEBUG=1
```

Or in `config/packages/doctrine.yaml`:

```yaml
doctrine:
  dbal:
    logging: true
```

Run SQL Manually

```
# Query the object table
bin/console doctrine:query:sql "SELECT * FROM object_query_1 WHERE
o_key='ProductX'"

# Check how many objects exist
bin/console doctrine:query:sql "SELECT COUNT(*) FROM objects WHERE
o_type='object'"
```

MySQL Slow Query Log

```ini
# /etc/mysql/my.cnf
slow_query_log = 1
slow_query_log_file = /var/log/mysql/slow.log
long_query_time = 1    ; log queries taking > 1 second
```

---

4. Common Errors & Fixes

Class Not Found (`DataObject\Product`)

Symptom: `Class 'Pimcore\Model\DataObject\Product' not found`

Fix:

```
bin/console pimcore:deployment:classes-rebuild --create-classes
composer dump-autoload --optimize
bin/console cache:clear
```

404 After Deployment

Symptom: Routes return 404 after deploying to a new server.

Checklist:

```
# 1. Rebuild class definitions
bin/console pimcore:deployment:classes-rebuild -c -d

# 2. Clear all caches
bin/console cache:clear
bin/console pimcore:cache:clear --all

# 3. Check document root points to /public
# Nginx config: root /var/www/pimcore/public;

# 4. Fix file permissions
sudo chown -R www-data:www-data var/ public/var/
sudo chmod -R 775 var/ public/var/
```

Database Connection Error (SQLSTATE[HY000] [2002])

Fix:

```
# Check if DB container is running
docker compose ps

# Check DB logs
docker compose logs db

# Verify .env DATABASE_URL
cat .env.local | grep DATABASE
# Should be: mysql://user:pass@db:3306/pimcore (not localhost!)
```

Cache Issues (Stale Data)

```
# Clear everything
bin/console cache:clear
bin/console pimcore:cache:clear --all

# For Redis
redis-cli FLUSHDB
```

```
# Rebuild assets
bin/console assets:install public --symlink
```

File Permission Errors

```
# Fix write permissions
sudo chown -R www-data:www-data var/ public/var/
sudo chmod -R 775 var/ public/var/ config/

# Or inside Docker
docker compose exec php bash -c "chown -R www-data:www-data var public/var"
```

Missing Migrations

```
bin/console doctrine:migrations:migrate --no-interaction --env=prod
```

---

5. Debugging Permissions

Symptom: Editor cannot see objects in the tree.

Fix:

1. Check roles & users: Settings → Users / Roles
2. Verify the folder has **"List children"** permission enabled for the role
3. Permissions don't automatically cascade if "list children" is unchecked

```
# List all users
bin/console pimcore:user:list

# Check security configuration
bin/console security:check
```

---

6. Debug Console Commands

```
# List all routes
bin/console debug:router

# Show all registered services
bin/console debug:container

# List all events
bin/console debug:event-dispatcher

# List enabled bundles
bin/console debug:bundle

# Check class definitions
bin/console pimcore:deployment:classes-rebuild --dump-classes

# Test your API
curl -v http://localhost/api/products

# Check system requirements
bin/console pimcore:system:requirements:check
```

---

7. Debugging in Docker

```
# Get a shell inside PHP container
docker compose exec php bash

# Run Symfony console
docker compose exec php bin/console cache:clear

# Check PHP configuration
docker compose exec php php -i | grep -i opcache

# Check Redis connection
docker compose exec php php -r "new Redis(); $r = new Redis(); $r-
>connect('redis', 6379); echo $r->ping();"

# Check running processes
docker compose ps

# Full log output
docker compose logs --tail=100 php
```

```
# Restart a container
docker compose restart php
```

---

8. Xdebug Setup (IDE Debugging)

Install Xdebug:

```
sudo apt install php8.3-xdebug
```

Config in `php.ini` :

```ini
zend_extension=xdebug.so
xdebug.mode=debug
xdebug.start_with_request=yes
xdebug.client_host=host.docker.internal   ; from inside Docker
xdebug.client_port=9003
xdebug.idekey=PHPSTORM
```

In VS Code (PHP Debug extension):

```json
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Listen for Xdebug",
            "type": "php",
            "request": "launch",
            "port": 9003,
            "pathMappings": {
                "/var/www/html": "${workspaceFolder}"
            }
        }
    ]
}
```

Quick Debugging Checklist

When Pimcore misbehaves, follow this sequence:

```
1. Check logs:       tail -f var/log/dev.log
2. Clear cache:      bin/console cache:clear && bin/console pimcore:cache:clear
3. Rebuild classes:  bin/console pimcore:deployment:classes-rebuild -c
4. Check routes:     bin/console debug:router | grep your-route
5. Check DB:         docker compose ps && docker compose logs db
6. Check Symfony profiler for slow queries
7. Check file permissions:  ls -la var/ public/var/
8. Run requirements check:  bin/console pimcore:system:requirements:check
```

Chapter 16: Testing

Overview

Testing in Pimcore follows Symfony's testing philosophy: **PHPUnit** for unit tests, **Symfony's WebTestCase** for functional/controller tests, and integration tests when you need to interact with actual Pimcore objects and the database.

---

1. PHPUnit Setup

```
composer require --dev phpunit/phpunit symfony/phpunit-bridge
```

`phpunit.xml.dist` at project root:

xml

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit
    bootstrap="vendor/autoload.php"
    colors="true"
    failOnWarning="true"
    stopOnFailure="false"
    verbose="true"
>
    <testsuites>
```

```
        <testsuite name="Application Test Suite">
            <directory>tests</directory>
        </testsuite>
    </testsuites>

    <php>
        <env name="APP_ENV" value="test"/>
        <env name="APP_DEBUG" value="1"/>
        <env name="PIMCORE_ENVIRONMENT" value="test"/>
        <env name="DATABASE_URL"
value="mysql://root:@127.0.0.1:3306/pimcore_test"/>
    </php>
</phpunit>
```

---

2. Unit Tests

Unit tests are fast — they don't need a database or HTTP client.

The Service

php

```php
// src/Service/GreetingService.php
namespace App\Service;

class GreetingService
{
    public function greet(string $name): string
    {
        return "Hello, " . ucfirst($name) . "!";
    }
}
```

The Test

php

```php
// tests/Service/GreetingServiceTest.php
namespace App\Tests\Service;

use App\Service\GreetingService;
use PHPUnit\Framework\TestCase;
```

```php
class GreetingServiceTest extends TestCase
{
    public function testGreetReturnsFormattedMessage(): void
    {
        $service = new GreetingService();
        $result = $service->greet('sandeep');

        $this->assertSame("Hello, Sandeep!", $result);
    }

    public function testGreetHandlesEmptyName(): void
    {
        $service = new GreetingService();
        $result = $service->greet('');

        $this->assertStringStartsWith("Hello, ", $result);
    }
}
```

---

3. Functional Tests (Controllers & APIs)

Functional tests spin up the Symfony kernel and make real HTTP requests through the test client.

The Controller

php

```php
// src/Controller/Api/ProductController.php
namespace App\Controller\Api;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\Routing\Annotation\Route;

class ProductController extends AbstractController
{
    #[Route('/api/test', name: 'api_test', methods: ['GET'])]
    public function test(): JsonResponse
    {
        return $this->json(['message' => 'API works']);
    }
}
```

The Functional Test

```php
// tests/Controller/ProductControllerTest.php
namespace App\Tests\Controller;

use Symfony\Bundle\FrameworkBundle\Test\WebTestCase;

class ProductControllerTest extends WebTestCase
{
    public function testApiEndpointReturnsJson(): void
    {
        $client = static::createClient();
        $client->request('GET', '/api/test');

        $this->assertResponseIsSuccessful();
        $this->assertResponseHeaderSame('content-type', 'application/json');
        $this->assertJsonStringEqualsJsonString(
            json_encode(['message' => 'API works']),
            $client->getResponse()->getContent()
        );
    }

    public function testProductListEndpoint(): void
    {
        $client = static::createClient();
        $client->request('GET', '/api/products?page=1&limit=5');

        $this->assertResponseIsSuccessful();
        $data = json_decode($client->getResponse()->getContent(), true);

        $this->assertArrayHasKey('products', $data);
        $this->assertArrayHasKey('count', $data);
        $this->assertArrayHasKey('page', $data);
    }

    public function testProductNotFound(): void
    {
        $client = static::createClient();
        $client->request('GET', '/product/99999999');

        $this->assertResponseStatusCodeSame(404);
    }
}
```

4. Running Tests

```
# Run all tests
bin/phpunit

# Run a specific test file
bin/phpunit tests/Service/GreetingServiceTest.php

# Run a specific test method
bin/phpunit --filter testGreetReturnsFormattedMessage

# Run with coverage report (requires Xdebug or PCOV)
XDEBUG_MODE=coverage bin/phpunit --coverage-html var/coverage

# Verbose output
bin/phpunit --testdox
```

Open coverage report:

```
xdg-open var/coverage/index.html
```

---

5. Testing Commands

php

```php
// tests/Command/ImportProductsCommandTest.php
namespace App\Tests\Command;

use Symfony\Bundle\FrameworkBundle\Console\Application;
use Symfony\Bundle\FrameworkBundle\Test\KernelTestCase;
use Symfony\Component\Console\Tester\CommandTester;

class ImportProductsCommandTest extends KernelTestCase
{
    public function testCommandFailsWithMissingCsv(): void
    {
        $kernel = static::createKernel();
        $application = new Application($kernel);

        $command = $application->find('app:import-products');
        $tester = new CommandTester($command);
```

```
        $tester->execute([]);

        $this->assertStringContainsString('CSV not found', $tester->getDisplay());
    }
}
```

---

6. CI/CD Integration (GitHub Actions)

.github/workflows/tests.yml

```yaml
name: Pimcore Tests

on: [push, pull_request]

jobs:
  tests:
    runs-on: ubuntu-latest
    services:
      mysql:
        image: mysql:8
        env:
          MYSQL_DATABASE: pimcore_test
          MYSQL_USER: root
          MYSQL_ALLOW_EMPTY_PASSWORD: yes
        ports:
          - 3306:3306
        options: >-
          --health-cmd="mysqladmin ping --silent"
          --health-interval=10s
          --health-timeout=5s
          --health-retries=3

      redis:
        image: redis:7
        ports:
          - 6379:6379

    steps:
      - uses: actions/checkout@v4
```

```
      - name: Setup PHP
        uses: shivammathur/setup-php@v2
        with:
          php-version: '8.3'
          extensions: mbstring, intl, pdo_mysql, redis
          coverage: xdebug

      - name: Install Composer dependencies
        run: composer install --prefer-dist --no-progress

      - name: Setup .env.test
        run: |
          cp .env .env.test.local
          echo "DATABASE_URL=mysql://root:@127.0.0.1:3306/pimcore_test" >>
.env.test.local
          echo "APP_ENV=test" >> .env.test.local

      - name: Setup database
        run: |
          bin/console doctrine:database:create --env=test --if-not-exists
          bin/console doctrine:migrations:migrate --env=test --no-interaction

      - name: Run tests
        run: bin/phpunit --testdox

      - name: Upload coverage
        if: always()
        uses: codecov/codecov-action@v4
```

---

7. Testing Best Practices

Type	When to Use	Speed
Unit tests	Services, helper classes, business logic (no DB)	Fast (milliseconds)
Functional tests	Controllers, API responses, routes	Medium (seconds)
Integration tests	Full Pimcore stack: create objects, fetch via API	Slow (requires DB)

Rules

- Write **unit tests for services** — fast, no DB dependency
- Write **functional tests for controllers/APIs** — via Symfony's test client
- Keep **Pimcore-specific code** (DataObjects, Assets) in integration tests — these need DB + Pimcore bootstrap
- Use a **separate test database** (`pimcore_test`) and reset before test runs
- Run tests in **CI** (GitHub Actions, GitLab CI, Jenkins)
- Mock external dependencies (HTTP calls, email, queues) in unit/functional tests

Useful Assertions

```php
$this->assertResponseIsSuccessful();          // 2xx status
$this->assertResponseStatusCodeSame(404);     // exact status code
$this->assertResponseHeaderSame('content-type', 'application/json');
$this->assertJsonStringEqualsJsonString($expected, $actual);
$this->assertStringContainsString('text', $response->getContent());
$this->assertCount(5, $data['products']);
```

Chapter 17: Deployment & CI/CD

Overview

Pimcore deployments require careful orchestration: code, class definitions, database migrations, caches, and assets all need to be handled correctly. This chapter covers a safe, repeatable deployment workflow.

---

1. Environment Variables

Never hardcode credentials. Store environment-specific values in `.env.local` (never commit to Git).

`.env` (committed — safe defaults):

```dotenv
APP_ENV=prod
APP_DEBUG=0
```

```
DATABASE_URL="mysql://user:pass@mysql:3306/pimcore_prod"
PIMCORE_ENVIRONMENT=prod
```

`.env.local` (NOT committed — overrides per server):

```dotenv
DATABASE_URL="mysql://real_user:real_pass@mysql:3306/pimcore_prod"
PIMCORE_IMAGE_PROXY_SECRET=xxxx
REDIS_URL=redis://redis:6379
```

> **In Docker/Kubernetes:** Inject env vars through **Secrets**, not `.env` files.

---

2. Database Changes

Pimcore stores data structure in **Class Definitions** (JSON files in `var/classes` or `config/pimcore/classes`).

Rules:

1. Always commit class JSON files to Git
2. Use `classes-rebuild` to sync DB schema after deploying class changes
3. Use Doctrine migrations for custom tables/entities

```
# After deploying new class definitions
bin/console pimcore:deployment:classes-rebuild --create-classes --delete-
removed-classes

# Run Doctrine migrations for custom tables
bin/console doctrine:migrations:migrate --no-interaction --env=prod
```

> **Golden rule:** No manual DB changes. Everything through migrations or class definitions.

---

3. Assets (Files, Images, Documents)

- Assets live in `/var/assets` (or external storage)
- In CI/CD: **don't overwrite!** Use shared storage (NFS, S3, Azure Blob)
- For large deployments: serve assets via CDN

```
# If using S3:
# Configure in config/packages/flysystem.yaml
# Use pimcore/cloud-storage bundle
```

---

4. Full Deployment Sequence

Run these commands in order on every deploy:

```
#!/bin/bash
set -e

echo "=== Step 1: Install Composer dependencies ==="
composer install --no-dev --optimize-autoloader --prefer-dist

echo "=== Step 2: Sync class definitions to DB ==="
bin/console pimcore:deployment:classes-rebuild --create-classes --delete-
removed-classes

echo "=== Step 3: Run database migrations ==="
bin/console doctrine:migrations:migrate --no-interaction --env=prod

echo "=== Step 4: Clear and warm up caches ==="
bin/console cache:clear --env=prod
bin/console cache:warmup --env=prod
bin/console pimcore:cache:clear --all

echo "=== Step 5: Install bundle assets (if custom bundles added) ==="
bin/console assets:install public --symlink --env=prod

echo "=== Done ==="
```

---

5. Git Branching Strategy

```
main          →  production (protected, deployment by tag/release)
develop       →  staging (auto-deploys on push)
feature/xxx   →  development (merge into develop via PR)
hotfix/xxx    →  emergency fix (merge into main AND develop)
```

6. CI/CD Pipeline (GitHub Actions)

.github/workflows/deploy.yml

yaml

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [develop, main]
  pull_request:
    branches: [develop]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      mysql:
        image: mysql:8
        env:
          MYSQL_DATABASE: pimcore_test
          MYSQL_USER: root
          MYSQL_ALLOW_EMPTY_PASSWORD: yes
        ports:
          - 3306:3306

    steps:
      - uses: actions/checkout@v4

      - name: Setup PHP
        uses: shivammathur/setup-php@v2
        with:
          php-version: '8.3'
          extensions: mbstring, intl, pdo_mysql

      - name: Install dependencies
        run: composer install --prefer-dist --no-progress

      - name: Run tests
        run: bin/phpunit --testdox

  deploy-staging:
    needs: test
    runs-on: ubuntu-latest
```

```
    if: github.ref == 'refs/heads/develop'
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to staging
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.STAGING_HOST }}
          username: deploy
          key: ${{ secrets.STAGING_SSH_KEY }}
          script: |
            cd /var/www/pimcore-staging
            git pull origin develop
            composer install --no-dev --optimize-autoloader
            bin/console pimcore:deployment:classes-rebuild -c -d
            bin/console doctrine:migrations:migrate --no-interaction --env=prod
            bin/console cache:clear --env=prod
            bin/console pimcore:cache:clear --all

  deploy-production:
    needs: test
    runs-on: ubuntu-latest
    if: startsWith(github.ref, 'refs/tags/v')
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to production
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.PROD_HOST }}
          username: deploy
          key: ${{ secrets.PROD_SSH_KEY }}
          script: |
            cd /var/www/pimcore-prod

            # Backup DB before migration
            mysqldump -u pimcore -p$DB_PASS pimcore_prod >
/backups/pimcore_$(date +%Y%m%d_%H%M%S).sql

            git pull origin main
            composer install --no-dev --optimize-autoloader
            bin/console pimcore:deployment:classes-rebuild -c -d
            bin/console doctrine:migrations:migrate --no-interaction --env=prod
            bin/console cache:clear --env=prod
```

```
bin/console cache:warmup --env=prod
bin/console pimcore:cache:clear --all
```

---

7. Zero-Downtime Deployment with Symlinks

For production deployments that must have zero downtime:

```bash
#!/bin/bash
# deploy-zero-downtime.sh

DEPLOY_DIR="/var/www/releases/$(date +%Y%m%d_%H%M%S)"
CURRENT_LINK="/var/www/pimcore"
SHARED_DIR="/var/www/shared"

# Create new release directory
mkdir -p "$DEPLOY_DIR"
git clone --depth 1 git@github.com:company/pimcore.git "$DEPLOY_DIR"

cd "$DEPLOY_DIR"

# Symlink shared directories (assets, var/assets persist across deploys)
ln -nfs "$SHARED_DIR/var/assets" "$DEPLOY_DIR/var/assets"
ln -nfs "$SHARED_DIR/.env.local" "$DEPLOY_DIR/.env.local"

# Install dependencies
composer install --no-dev --optimize-autoloader

# Run migrations (before switching symlink)
bin/console pimcore:deployment:classes-rebuild -c -d
bin/console doctrine:migrations:migrate --no-interaction --env=prod
bin/console cache:clear --env=prod
bin/console cache:warmup --env=prod

# Switch symlink atomically (near-zero downtime)
ln -nfs "$DEPLOY_DIR" "$CURRENT_LINK"

# Reload PHP-FPM
sudo systemctl reload php8.3-fpm

echo "Deployed to $DEPLOY_DIR"
```

8. Extra Safety Measures

Practice	Command / Config
DB backup before migration	`mysqldump` before `doctrine:migrations:migrate`
Verify migration before running	`bin/console doctrine:migrations:status`
Feature flags	Environment variables or DB-toggle for risky features
Health check endpoint	Custom route returning 200 + app version
Rollback plan	Keep last 3 releases, symlink back if deploy fails
Audit logs	Keep Pimcore admin logs enabled

Deployment in One Sentence

> Commit code + class JSONs → push to develop → CI runs tests → deploy to staging → QA review → merge/tag release → deploy to production with composer install + classes-rebuild + migrations + cache clear, keeping assets in shared storage.

Chapter 18: Kubernetes Setup & Debugging

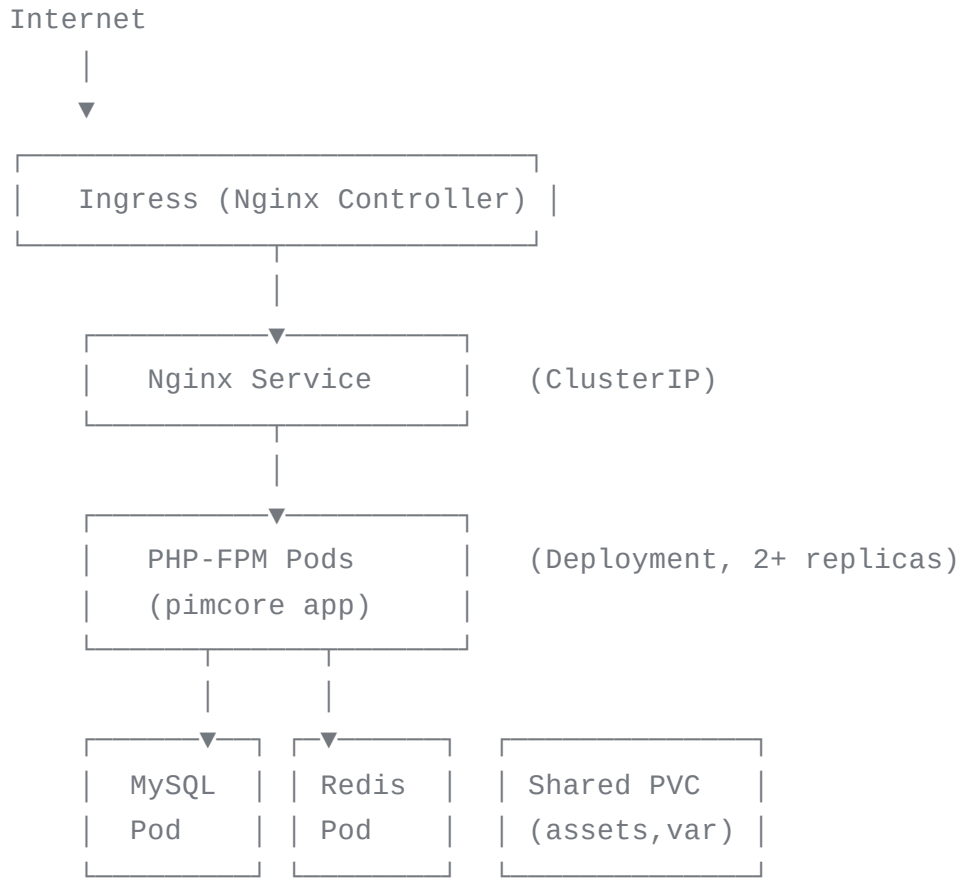
Overview

Running Pimcore on Kubernetes (K8s) gives you:

- **Horizontal scaling** — multiple PHP-FPM pods
- **High availability** — automatic restarts, rolling updates
- **Infrastructure as code** — all config in YAML files
- **Environment isolation** — dev, staging, prod on same cluster

Pimcore on Kubernetes requires: a PHP-FPM app pod, a MySQL pod (or external RDS), a Redis pod, a shared persistent volume for assets, and an Nginx ingress controller.

Architecture



Step 1: Docker Image for Pimcore

Create a production Docker image:

`Dockerfile`:

dockerfile

```
FROM pimcore/pimcore:php8.3-latest

WORKDIR /var/www/html

# Copy project files
COPY . .
```

```
# Install Composer dependencies (no dev)
RUN composer install --no-dev --optimize-autoloader --prefer-dist

# Set permissions
RUN chown -R www-data:www-data var/ public/var/ && \
    chmod -R 775 var/ public/var/

EXPOSE 9000

CMD ["php-fpm"]
```

Nginx sidecar `nginx.conf`:

nginx

```
server {
    listen 80;
    server_name _;
    root /var/www/html/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php$is_args$args;
    }

    location ~ \.php$ {
        fastcgi_pass 127.0.0.1:9000;
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
        fastcgi_read_timeout 300;
    }

    location ~* \.(jpg|jpeg|png|gif|ico|css|js|woff2)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

Step 2: Namespace & ConfigMap

```
kubectl create namespace pimcore
```

k8s/configmap.yaml :

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: pimcore-config
  namespace: pimcore
data:
  APP_ENV: "prod"
  APP_DEBUG: "0"
  PIMCORE_ENVIRONMENT: "prod"
  REDIS_HOST: "redis-service"
  REDIS_PORT: "6379"
  MYSQL_HOST: "mysql-service"
  MYSQL_DATABASE: "pimcore"
  MYSQL_USER: "pimcore"
```

Step 3: Secrets

k8s/secret.yaml :

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: pimcore-secrets
  namespace: pimcore
type: Opaque
stringData:
  MYSQL_PASSWORD: "your_secure_password"
  DATABASE_URL: "mysql://pimcore:your_secure_password@mysql-
service:3306/pimcore"
  APP_SECRET: "your_symfony_app_secret_32chars"
  PIMCORE_IMAGE_PROXY_SECRET: "your_proxy_secret"
```

```
kubectl apply -f k8s/secret.yaml -n pimcore
```

Step 4: Persistent Volume for Assets

Pimcore assets (`var/assets`) must be **shared across all PHP pods** — use a ReadWriteMany PVC.

`k8s/pvc.yaml` :

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pimcore-assets-pvc
  namespace: pimcore
spec:
  accessModes:
    - ReadWriteMany        # Required for multiple pods to share
  storageClassName: nfs    # Use NFS, EFS (AWS), or Filestore (GCP)
  resources:
    requests:
      storage: 50Gi
```

> **Important:** Use a `ReadWriteMany` storage class. Options:
>
> - On-premise: NFS server with `nfs-subdir-external-provisioner`
> - AWS: EFS with `aws-efs-csi-driver`
> - GCP: Filestore
> - Azure: Azure Files

Step 5: MySQL Deployment

`k8s/mysql.yaml` :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
  namespace: pimcore
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
```

```
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          env:
            - name: MYSQL_DATABASE
              valueFrom:
                configMapKeyRef:
                  name: pimcore-config
                  key: MYSQL_DATABASE
            - name: MYSQL_USER
              valueFrom:
                configMapKeyRef:
                  name: pimcore-config
                  key: MYSQL_USER
            - name: MYSQL_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: pimcore-secrets
                  key: MYSQL_PASSWORD
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: pimcore-secrets
                  key: MYSQL_PASSWORD
          ports:
            - containerPort: 3306
          volumeMounts:
            - name: mysql-data
              mountPath: /var/lib/mysql
      volumes:
        - name: mysql-data
          persistentVolumeClaim:
            claimName: mysql-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: mysql-service
  namespace: pimcore
spec:
  selector:
    app: mysql
  ports:
    - port: 3306
```

```
    targetPort: 3306
  clusterIP: None   # Headless service for stable DNS
```

Step 6: Redis Deployment

k8s/redis.yaml :

yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: pimcore
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          ports:
            - containerPort: 6379
          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "512Mi"
              cpu: "500m"
---
apiVersion: v1
kind: Service
metadata:
  name: redis-service
  namespace: pimcore
spec:
  selector:
```

```
    app: redis
  ports:
    - port: 6379
      targetPort: 6379
```

Step 7: Pimcore App Deployment

k8s/pimcore-app.yaml :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pimcore-app
  namespace: pimcore
spec:
  replicas: 2          # Scale horizontally
  selector:
    matchLabels:
      app: pimcore-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0   # Zero-downtime rolling updates
  template:
    metadata:
      labels:
        app: pimcore-app
    spec:
      initContainers:
        # Run migrations and class rebuild before the app starts
        - name: pimcore-init
          image: your-registry/pimcore:latest
          command: ["/bin/sh", "-c"]
          args:
            - |
              bin/console pimcore:deployment:classes-rebuild -c -d
              bin/console doctrine:migrations:migrate --no-interaction --
env=prod
              bin/console cache:clear --env=prod
              bin/console cache:warmup --env=prod
          envFrom:
```

```
        - configMapRef:
            name: pimcore-config
        - secretRef:
            name: pimcore-secrets
      volumeMounts:
        - name: pimcore-assets
          mountPath: /var/www/html/var/assets
        - name: pimcore-cache
          mountPath: /var/www/html/var/cache

  containers:
    # Nginx container (serves static files, proxies PHP)
    - name: nginx
      image: nginx:1.25-alpine
      ports:
        - containerPort: 80
      volumeMounts:
        - name: nginx-config
          mountPath: /etc/nginx/conf.d
        - name: pimcore-assets
          mountPath: /var/www/html/var/assets
          readOnly: true

    # PHP-FPM container (runs Pimcore)
    - name: php-fpm
      image: your-registry/pimcore:latest
      envFrom:
        - configMapRef:
            name: pimcore-config
        - secretRef:
            name: pimcore-secrets
      ports:
        - containerPort: 9000
      volumeMounts:
        - name: pimcore-assets
          mountPath: /var/www/html/var/assets
        - name: pimcore-cache
          mountPath: /var/www/html/var/cache
      resources:
        requests:
          memory: "512Mi"
          cpu: "250m"
        limits:
          memory: "2Gi"
          cpu: "1000m"
      livenessProbe:
```

```
            exec:
              command: ["php-fpm-healthcheck"]
            initialDelaySeconds: 30
            periodSeconds: 10
          readinessProbe:
            exec:
              command: ["php-fpm-healthcheck", "--accepted-conn=1"]
            initialDelaySeconds: 5
            periodSeconds: 5

      volumes:
        - name: pimcore-assets
          persistentVolumeClaim:
            claimName: pimcore-assets-pvc
        - name: pimcore-cache
          emptyDir: {}      # Cache is ephemeral per pod (Redis handles actual
cache)
        - name: nginx-config
          configMap:
            name: nginx-config
---
apiVersion: v1
kind: Service
metadata:
  name: pimcore-service
  namespace: pimcore
spec:
  selector:
    app: pimcore-app
  ports:
    - port: 80
      targetPort: 80
```

Step 8: Ingress (External Access)

k8s/ingress.yaml :

yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: pimcore-ingress
```

```
  namespace: pimcore
  annotations:
    kubernetes.io/ingress.class: nginx
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/proxy-body-size: "100m"     # allow large file
uploads
    nginx.ingress.kubernetes.io/proxy-read-timeout: "300"
spec:
  tls:
    - hosts:
        - pimcore.yourdomain.com
      secretName: pimcore-tls
  rules:
    - host: pimcore.yourdomain.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: pimcore-service
                port:
                  number: 80
```

Step 9: Deploy Everything

```
# Apply all manifests
kubectl apply -f k8s/ -n pimcore

# Watch rollout
kubectl rollout status deployment/pimcore-app -n pimcore

# Check all pods
kubectl get pods -n pimcore
```

Debugging in Kubernetes

Get a Shell Inside a Pod

```
# Get pod names
kubectl get pods -n pimcore

# Shell into PHP-FPM container
kubectl exec -it <pod-name> -n pimcore -c php-fpm -- bash

# Run Pimcore console inside pod
kubectl exec -it <pod-name> -n pimcore -c php-fpm -- bin/console cache:clear
kubectl exec -it <pod-name> -n pimcore -c php-fpm -- bin/console
pimcore:system:requirements:check
```

View Logs

```
# All logs from pimcore-app pods
kubectl logs -l app=pimcore-app -n pimcore --all-containers

# Follow logs from specific pod
kubectl logs -f <pod-name> -n pimcore -c php-fpm

# Previous pod logs (after a crash)
kubectl logs <pod-name> -n pimcore -c php-fpm --previous

# View Pimcore's own log files inside pod
kubectl exec -it <pod-name> -n pimcore -c php-fpm -- tail -f var/log/prod.log
```

Debug Pod Startup Issues

```
# Describe pod (shows events, failed probes, image pull errors)
kubectl describe pod <pod-name> -n pimcore

# Check if init container ran successfully
kubectl logs <pod-name> -n pimcore -c pimcore-init

# Check all events in namespace
kubectl get events -n pimcore --sort-by='.lastTimestamp'
```

Debug Database Connection

```
# Exec into app pod and test MySQL
kubectl exec -it <pod-name> -n pimcore -c php-fpm -- \
  bin/console doctrine:query:sql "SELECT 1"

# Check if MySQL service is reachable
kubectl exec -it <pod-name> -n pimcore -c php-fpm -- \
  nc -zv mysql-service 3306
```

Debug Redis Connection

```
kubectl exec -it <pod-name> -n pimcore -c php-fpm -- \
  php -r "
    \$r = new Redis();
    \$r->connect('redis-service', 6379);
    echo \$r->ping() . PHP_EOL;
  "
```

Check Persistent Volume

```
# Check PVC status
kubectl get pvc -n pimcore

# Check if assets directory is mounted and writable
kubectl exec -it <pod-name> -n pimcore -c php-fpm -- \
  ls -la var/assets/
```

Horizontal Pod Autoscaler (HPA)

Scale PHP pods automatically based on CPU:

k8s/hpa.yaml:

yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: pimcore-hpa
  namespace: pimcore
spec:
  scaleTargetRef:
```

```
    apiVersion: apps/v1
    kind: Deployment
    name: pimcore-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
```

Helm Chart (Optional — Recommended for Production)

For repeatable deployments across environments, use a **Helm chart**:

```
# Create a Helm chart
helm create pimcore-chart

# Values file for staging
helm install pimcore ./pimcore-chart -f values.staging.yaml -n pimcore

# Values file for production
helm install pimcore ./pimcore-chart -f values.prod.yaml -n pimcore

# Upgrade a release
helm upgrade pimcore ./pimcore-chart -f values.prod.yaml -n pimcore
```

Common Kubernetes Issues for Pimcore

Issue	Cause	Fix
Pods in `CrashLoopBackOff`	Migration failed / DB not ready	Check init container logs
Assets not shared between pods	PVC not ReadWriteMany	Use NFS/EFS storage class
502 Bad Gateway	PHP-FPM not ready	Check readinessProbe / increase `initialDelaySeconds`
Cache inconsistency across pods	Filesystem cache used	Switch to Redis cache adapter
Large file upload fails	Nginx body size limit	Set `nginx.ingress.kubernetes.io/proxy-body-size: "100m"`
OOM killed pods	PHP memory limit too low	Increase pod memory limits
Slow cold start	No OPcache preloading	Add OPcache preload config

---

Kubernetes Cheatsheet

```
# Get all resources in namespace
kubectl get all -n pimcore

# Watch pods
kubectl get pods -n pimcore -w

# Scale deployment
kubectl scale deployment pimcore-app --replicas=4 -n pimcore

# Rolling restart (pick up new image or config)
kubectl rollout restart deployment/pimcore-app -n pimcore

# View rollout history
kubectl rollout history deployment/pimcore-app -n pimcore
```

```
# Rollback to previous version
kubectl rollout undo deployment/pimcore-app -n pimcore

# Port-forward to test locally
kubectl port-forward svc/pimcore-service 8080:80 -n pimcore
# Then open: http://localhost:8080
```

Chapter 19: Contributing to Pimcore

Overview

Contributing to Pimcore open source is a great way to level up your skills, build reputation, and give back to the community. This chapter walks through the entire contribution cycle — from finding an issue to getting your PR merged.

---

1. Find "Good First Issues"

- **GitHub:** `https://github.com/pimcore/pimcore`
- Go to **Issues** → **Labels** → `good first issue` or `help wanted`
- Pick small, focused issues: typo fix, docs improvement, small bug, minor feature

---

2. Fork & Clone

```
# Fork via GitHub UI, then:
git clone https://github.com/<your-username>/pimcore.git
cd pimcore

# Add upstream for syncing
git remote add upstream https://github.com/pimcore/pimcore.git
```

---

3. Set Up Core Dev Environment

```
# Install dependencies
composer install

# Copy test config
cp .env.test.dist .env.test

# Create test DB
mysql -u root -p -e "CREATE DATABASE pimcore_test;"

# Run DB schema init
bin/console doctrine:migrations:migrate --env=test --no-interaction

# Run full test suite to confirm everything works
vendor/bin/phpunit
```

---

4. Create a Branch & Make Changes

```
# Always branch from the current development branch (e.g., 10.x or 12.x)
git checkout -b fix/typo-in-product-service

# Make your changes
# ...

# Verify code quality
vendor/bin/phpunit
bin/console lint:container
bin/console lint:twig

# Commit with a clear message
git commit -m "Fix typo in ProductService docblock"
```

---

5. Submit a Pull Request

```
git push origin fix/typo-in-product-service
```

On GitHub → your fork → GitHub suggests creating a PR.

- **Base branch:** `pimcore/pimcore:10.x` (or current dev branch)
- **Compare branch:** your `fix/...`

---

6. PR Description Template

markdown

```
## Changes in this PR
- Fixed a typo in ProductService docblock.
- Added missing test case for product slug generation.

## Why this is needed
Improves code readability and prevents confusion for new developers.

## Related Issues
Closes #12345

## Checklist
- [x] Tests pass locally (`vendor/bin/phpunit`)
- [x] No BC (backward compatibility) breaks
- [x] Code follows Pimcore coding style (PSR-12)
- [x] Docs updated if needed
```

---

7. After PR Submission

1. **CI runs automatically** → fix any failing checks
2. **Maintainers review** → may request changes → update your branch and push again
3. **Approved** → **Merged** 🎉

Best Practices

```
# Keep branch in sync with upstream
git fetch upstream
git rebase upstream/10.x
```

```
# Fix code style
composer cs-fix   # or vendor/bin/php-cs-fixer fix

# Check for BC breaks
bin/console debug:container
```

- Always branch from `10.x` (not `main`)
- Keep commits small and focused
- Use **PSR-12 coding style**
- Write tests for your changes
- Keep PRs focused — one change per PR

Chapter 20: Learning Roadmap — 5 Weeks to Pimcore Proficiency

Overview

This is a structured, hands-on 5-week plan that takes you from zero → a real open-source contribution, with concrete daily tasks and clear success criteria.

Week 1: Install & Basics

Goal: Have Pimcore running locally, know the Admin UI, commit first project code.

Day	Task
Day 1–2	Install Pimcore with Docker + Composer. Access Admin UI at `/admin`. Explore Data Objects, Assets, Documents sections.
Day 3–4	Create a simple Product class (name, price, sku). Add a few objects manually.
Day 5	Explore Assets — upload images and PDFs. Try linking assets to your product.

Day	Task
Day 6–7	Play with Documents — create a CMS page, add text/images. Read Pimcore docs on Data Objects & Assets.

Success Criteria:

- Pimcore runs locally
- Can create/update a Product object with an image via Admin UI
- First Git commit: project bootstrapped

Week 2: Classes & Assets

Goal: Model richer data and handle media programmatically.

Day	Task
Day 1–2	Add fields: description, categories (relation), images (asset relation). Add localized fields for multi-language.
Day 3	Write a PHP script to create/update Product objects programmatically.
Day 4	Import 5 products from a small CSV using `fgetcsv` or `League\Csv`.
Day 5	Define a thumbnail preset in Admin UI (Settings → Thumbnails).
Day 6	Create a thumbnail variant programmatically via PHP.
Day 7	Document your Product data model (draw a diagram + notes).

Success Criteria:

- Product model with relations + localized fields exists
- CSV import script works
- Thumbnail variant displays correctly

Week 3: APIs & Import/Export

Goal: Expose and consume Pimcore data via APIs.

Day	Task
Day 1–2	Create a Symfony controller returning Products as JSON. Add pagination + filtering (`?page=2&category=shoes`).
Day 3	Test your API with curl and Postman.
Day 4–5	Enable Pimcore DataHub/GraphQL bundle. Write a GraphQL query to fetch products with images/categories.
Day 6	Write an export command: Products → CSV.
Day 7	Combine: fetch via API → save to CSV → re-import.

Success Criteria:

- REST endpoint works with pagination & filters
- GraphQL query returns product + image data
- Export + re-import cycle proven

Week 4: Admin Extensions & Testing

Goal: Extend Admin UI and write tests.

Day	Task
Day 1–2	Add a custom button in Product grid (e.g., "Export CSV").
Day 3–4	Build a small bundle with a custom admin page (e.g., product stats).
Day 5	Set up PHPUnit config. Write a unit test for a simple service.
Day 6–7	Write a functional test for your Product API endpoint.

Success Criteria:

- Custom button appears in Product grid
- Bundle loads a custom admin page
- PHPUnit runs with passing unit + functional tests

Week 5: Kubernetes & Contribution

Goal: Deploy to Kubernetes and submit your first Pimcore PR.

Day	Task
Day 1	Set up a local Kubernetes cluster (Minikube or k3s). Deploy Pimcore using the manifests from Chapter 18.
Day 2	Practice `kubectl logs`, `kubectl exec`, `kubectl describe pod` for debugging.
Day 3	Read Pimcore's Contributing Guidelines. Find a "good first issue" on GitHub.
Day 4–5	Fork Pimcore repo. Fix the issue. Run the full test suite.
Day 6	Open a PR against Pimcore's main dev branch. Follow the PR template.
Day 7	Respond to maintainer feedback if any.

Success Criteria:

- Pimcore running in Kubernetes locally
- Can debug pods and view logs
- PR open in Pimcore core repo with all CI tests passing

Summary Timeline

Week	Focus	Deliverable
Week 1	Setup & Admin UI basics	Pimcore running + Product object via Admin UI
Week 2	Data modeling & Assets	CSV import + thumbnails working
Week 3	APIs (REST + GraphQL)	Live API + export/import cycle
Week 4	Admin Extensions + Tests	Custom UI + PHPUnit passing

Week	Focus	Deliverable
Week 5	Kubernetes + Contribution	K8s deployment + PR submitted

---

Quick Reference: Most-Used Commands

```
# Development
docker compose up -d
docker compose exec php bin/console cache:clear
docker compose exec php bin/console pimcore:deployment:classes-rebuild -c
docker compose exec php bin/phpunit

# Kubernetes
kubectl get pods -n pimcore
kubectl logs -f <pod> -n pimcore -c php-fpm
kubectl exec -it <pod> -n pimcore -c php-fpm -- bash
kubectl rollout restart deployment/pimcore-app -n pimcore

# Pimcore Console
bin/console debug:router
bin/console debug:container
bin/console pimcore:system:requirements:check
bin/console app:import-products

# Git workflow
git checkout -b feature/my-feature
git fetch upstream && git rebase upstream/10.x
```

Recommended Reading

Resource	URL
Pimcore Docs	`docs.pimcore.com`
Pimcore GitHub	`github.com/pimcore/pimcore`

Resource	URL
Symfony Docs	`symfony.com/doc`
Pimcore Community	`pimcore.com/community`
Docker Docs	`docs.docker.com`
Kubernetes Docs	`kubernetes.io/docs`